

Memoria de Trabajo de Grado

SecureTicket

Solución B2B de reventa de entradas basada en
contratos inteligentes sobre Stellar/Soroban

Versión 1.0 — Mayo de 2026



Pontificia Universidad Javeriana, Bogotá

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas

Director: Rafael Vicente Páez Méndez, PhD

Autores (Grupo 12):

Juan Diego Arias Durán
Santiago Andrés Mesa Niño
Andrés Felipe Manosalva Garzón
Juan Camilo Carvajal Rodríguez

Trabajo de grado presentado como requisito parcial

para optar al título de Ingeniero de Sistemas

Semestre 2026-1

**PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS**

Rector de la Pontificia Universidad Javeriana

Padre Luis Fernando Múnera, S.J.

Decano de la Facultad de Ingeniería

Ing. Diego Alejandro Patiño Guevara, Sc.D.

Directora de la Carrera de Ingeniería de Sistemas

Ing. Andrea del Pilar Rueda Olarte, Ph.D.

Director del Trabajo de Grado

Ing. Rafael Vicente Páez Méndez, Ph.D.

Artículo 23 de la Resolución No. 13 de Junio de 1946

“La Universidad no se hace responsable de los conceptos emitidos por sus alumnos en sus proyectos de grado. Sólo velará porque no se publique nada contrario al dogma y la moral católica y porque los trabajos no contengan ataques o polémicas puramente personales. Antes bien, que se vean en ellos el anhelo de buscar la verdad y la justicia.”

Agradecimientos

Los autores expresan su agradecimiento al Ingeniero Rafael Vicente Páez Méndez, Ph.D., director del trabajo de grado, por su acompañamiento académico, sus revisiones periódicas y la orientación constante durante el desarrollo del proyecto.

Reconocen el aporte del Departamento de Ingeniería de Sistemas de la Pontificia Universidad Javeriana, así como el de los docentes evaluadores que aportaron observaciones a las versiones preliminares del documento.

Finalmente, los autores agradecen a sus familias por el apoyo durante el desarrollo del trabajo de grado.

Índice

	Página
Resumen	v
Abstract	vi
Glosario	vii
1. INTRODUCCIÓN	1
2. DESCRIPCIÓN GENERAL Y PLANTEAMIENTO DEL PROBLEMA	2
2.1. Oportunidad, Problema	2
2.1.1. Contexto de la Industria: De lo Global a lo Local	2
2.1.2. Formulación del Problema	2
2.1.3. Propuesta de Solución	2
2.1.4. Justificación de la Solución	3
2.2. Descripción del Proyecto	3
2.2.1. Objetivo General	3
2.2.2. Objetivos Específicos	3
2.2.3. Entregables, Estándares y Justificación	3
3. CONTEXTO DEL PROYECTO	5
3.1. Trasfondo	5
3.1.1. Introducción a la Tecnología Blockchain y Contratos Inteligentes	5
3.1.2. La Red Stellar	6
3.1.3. Protocolo de Consenso Stellar (SCP) y Acuerdo Bizantino Federado (FBA)	7
3.1.4. Tokens Estables (Stablecoins) y Bases Matemáticas del Consenso	8
3.1.5. Soroban, NFTs y tokens de pago	9
3.2. Análisis del Contexto	9
4. ANÁLISIS DEL PROBLEMA	11
4.1. Requerimientos	11
4.2. Restricciones	12
4.3. Especificación Funcional	12
5. DISEÑO DE LA SOLUCIÓN	15
5.1. Actores, roles y límites de confianza	15
5.2. Modelo del ticket NFT	16
5.2.1. Identificador y separación de datos sensibles	16
5.2.2. Estados y patrón burn/remint	16
5.3. Diseño de contratos inteligentes	17
5.3.1. factory_contract	17
5.3.2. event_contract	17
5.3.3. ticket_nft_contract	18
5.3.4. Errores tipados y eventos on-chain	18

5.4.	Arquitectura distribuida (tres nodos)	18
5.4.1.	Nodo web (frontend)	19
5.4.2.	Nodo API (backend)	19
5.4.3.	Nodo datos (PostgreSQL + indexador)	19
5.5.	Coleccionable Stellar Classic	20
6.	DESARROLLO DE LA SOLUCIÓN	21
6.1.	Implementación de contratos Soroban	21
6.1.1.	NFT del boleto: emisión, propiedad y versionado	21
6.1.2.	Pagos atómicos vía SAC	21
6.1.3.	Redención y control de acceso	22
6.2.	Backend Node.js + Stellar SDK	23
6.2.1.	API: usuarios, eventos, carrito, checkout y Soroban	24
6.2.2.	Indexador: ingesta de eventos	24
6.2.3.	Persistencia: historial, listados y auditoría	24
6.3.	Frontend web	25
6.4.	Despliegue distribuido	25
6.5.	Integración continua	26
6.6.	Pruebas	27
7.	RESULTADOS	28
7.1.	Metodología de medición	28
7.2.	Desempeño sobre testnet	28
7.3.	Cobertura y resultados de pruebas automatizadas	29
7.4.	Evaluación antifraude	29
7.5.	Análisis y discusión	30
7.6.	Evidencia funcional del prototipo	30
8.	CONCLUSIONES	34
8.1.	Análisis de Impacto del Proyecto	34
8.1.1.	Impacto en ingeniería de sistemas	34
8.1.2.	Impacto global, económico, ambiental y social	34
8.2.	Conclusiones y Trabajo Futuro	35
	Referencias	36
	A. ANEXOS	37

Índice de figuras

1.	Diagrama de casos de uso del sistema (4 actores, 6 casos de uso principales). . .	13
2.	Diagrama de contexto C4 (nivel 1) del sistema SecureTicket.	15
3.	Diagrama de estados del boleto: ACTIVE, USED y CANCELLED, con sus transiciones.	17
4.	Diagrama C4 de contenedores (nivel 2): nodos web, API y datos, y su acoplamiento con Soroban RPC y Horizon.	19
5.	Diagrama de secuencia: compra en reventa atómica (pago al vendedor, comisión a organizador y plataforma, <i>burn</i> de la versión vigente y <i>remint</i> de la nueva, todo en una sola transacción).	22
6.	Diagrama de secuencia: verificación del boleto en puerta (escaneo QR, validación en PostgreSQL y respaldo on-chain).	23
7.	Modelo entidad-relación de la base de datos relacional (esquema ticketing). .	25
8.	Diagrama de despliegue: frontend en Vercel, backend en Railway, base de datos PostgreSQL en Supabase y red Stellar testnet.	26
9.	Catálogo público de eventos en la pasarela B2B <i>TuTicket</i> , punto de entrada para la compra primaria.	31
10.	Listado de un boleto para reventa: el usuario fija el precio en COP, la interfaz muestra el equivalente en XLM a la cotización actual y descompone las comisiones (5 % organizador, 3 % plataforma) junto al monto neto a recibir. . .	31
11.	Confirmación de transacción Soroban en Freightier: el usuario revisa la operación construida por el backend (XDR proxy) y la firma localmente; las llaves privadas nunca abandonan la extensión.	32
12.	Vista “Mis Entradas” tras “Asegurar en Blockchain”: el boleto exhibe el badge de Secure Ticket y el QR firmado que se valida en puerta contra el contrato ticket_nft_contract	32
13.	Consola administrativa Secure Ticket: muestra los 76 eventos registrados y los 40 contratos desplegados a través del factory , junto a los accesos al <i>scanner</i> de redención y al panel de reglas P2P.	33

Índice de tablas

1.	Entregables y estándares aplicados.	4
2.	Análisis comparativo de soluciones del contexto.	10
3.	Requerimientos funcionales del sistema (resumen).	11
4.	Requerimientos no funcionales del sistema (resumen).	12
5.	Latencia de confirmación y costo nominal por operación sobre testnet.	28
6.	Distribución de pruebas unitarias sobre los contratos Soroban.	29
7.	Escenarios antifraude evaluados sobre el contrato.	29
8.	Anexos del Trabajo de Grado.	37

Resumen

Esta tesis presenta el diseño, la implementación y la validación de una solución B2B de reventa de entradas para eventos de entretenimiento, construida sobre contratos inteligentes Soroban en la red Stellar. El trabajo aborda tres problemas técnicos del mercado secundario: la duplicidad de códigos QR estáticos, la reventa concurrente del mismo activo a varios compradores y la ausencia de trazabilidad pública de la cadena de propiedad.

La propuesta materializa cada boleto como un token no fungible (NFT) identificado por la tupla (`ticket_root_id`, `version`), aplica un patrón *burn/remint* en cada operación de reventa para preservar la unicidad y ejecuta la liquidación de pagos y comisiones de forma atómica dentro de la misma invocación del contrato. El prototipo se despliega como una arquitectura de tres nodos (web, API y datos) sobre la red de prueba de Stellar, con doce eventos desplegados a través de un contrato *factory* (cada evento con su par `event_contract` + `ticket_nft_contract`) y un módulo indexador que mantiene la coherencia entre el ledger y una base de datos relacional.

La evaluación se realiza sobre testnet con métricas reproducibles de latencia de confirmación, costo nominal por operación, cobertura de pruebas unitarias y comportamiento del contrato ante escenarios antifraude. Los resultados permiten contrastar la solución frente a esquemas tradicionales de reventa en términos de tiempo de liquidación, costo por transacción y trazabilidad operacional.

Palabras clave: blockchain, Stellar, Soroban, contratos inteligentes, NFT, reventa de entradas, antifraude, ticketing digital.

Abstract

This thesis presents the design, implementation, and validation of a B2B ticket resale solution for entertainment events, built on Soroban smart contracts on the Stellar network. The work addresses three technical problems of the secondary market: duplication of static QR codes, concurrent resale of the same asset to multiple buyers, and the absence of public traceability of the ownership chain.

The proposal materializes each ticket as a non-fungible token (NFT) identified by the tuple (`ticket_root_id`, `version`), applies a *burn/remint* pattern on each resale operation to preserve uniqueness, and executes payment and commission settlement atomically within the same contract invocation. The prototype is deployed as a three-node architecture (web, API, data) on the Stellar testnet, with twelve events deployed through a factory contract (each event with its `event_contract` + `ticket_nft_contract` pair) and an indexer module that maintains consistency between the ledger and a relational database.

The evaluation is performed on testnet with reproducible metrics of confirmation latency, nominal cost per operation, unit-test coverage, and contract behavior under anti-fraud scenarios. The results enable comparison against traditional resale schemes in terms of settlement time, cost per transaction, and operational traceability.

Keywords: blockchain, Stellar, Soroban, smart contracts, NFT, ticket resale, anti-fraud, digital ticketing.

Glosario

A continuación se definen los términos técnicos utilizados a lo largo del documento. Las definiciones se proporcionan con el nivel de detalle suficiente para que el lector pueda interpretar la memoria sin requerir conocimiento previo sobre tecnologías de registro distribuido.

Conceptos generales de blockchain

Blockchain (cadena de bloques) Estructura de datos distribuida y enlazada criptográficamente en la que un conjunto de nodos mantiene una réplica del registro y acuerda el orden de las transacciones mediante un protocolo de consenso.

Ledger (libro mayor distribuido) Registro inmutable y compartido entre los nodos de la red. En Stellar, cada cierre de *ledger* consolida un conjunto de transacciones aprobadas por el protocolo de consenso.

Hash criptográfico Función que transforma un dato de longitud arbitraria en una cadena de tamaño fijo, de forma determinística y resistente a colisiones. Se utiliza para identificar bloques, transacciones y módulos de contratos.

Árbol de Merkle Estructura de árbol binario en la que cada nodo intermedio almacena el *hash* de la concatenación de sus hijos. Permite verificar la pertenencia de una transacción al bloque sin recorrer toda la lista.

Consenso Mecanismo por el cual los nodos de la red acuerdan el contenido de un nuevo bloque. Stellar utiliza el *Stellar Consensus Protocol* (SCP), distinto a Proof of Work y Proof of Stake.

Finalidad (*finality*) Propiedad por la cual una transacción confirmada no puede ser revertida. En Stellar la finalidad se alcanza en el cierre del *ledger*, en un intervalo de 3 a 5 segundos.

Mainnet / Testnet Redes operativas de Stellar. La *mainnet* es la red de producción con activos de valor real; la *testnet* es la red de pruebas con activos sin valor económico, utilizada en este trabajo.

Stellar y Soroban

Stellar Red pública descentralizada orientada a la transferencia de valor digital y a la emisión de activos tokenizados.

Soroban Plataforma de contratos inteligentes de Stellar, integrada desde el protocolo 20. Los contratos se escriben en Rust y se compilan al formato WebAssembly.

SCP (Stellar Consensus Protocol) Instancia del paradigma *Federated Byzantine Agreement*. Cada nodo declara su propia configuración de confianza (*quorum slices*) en lugar de depender de una membresía fija.

PoA (Proof of Agreement) Mecanismo de consenso implementado por el Protocolo de Consenso de Stellar (SCP). A diferencia de la Prueba de Trabajo (*Proof of Work*) o

la Prueba de Participación (*Proof of Stake*), la Prueba de Acuerdo permite lograr el consenso a través de una serie rápida de mensajes entre los participantes para confirmar transacciones, sin requerir soluciones de fuerza bruta matemática ni un alto consumo de energía [1, 2].

FBA (*Federated Byzantine Agreement*) Modelo de consenso bizantino en el que la membresía y los quóruns se definen de forma descentralizada por cada nodo.

Quórum / *Quorum slice* Subconjunto de nodos cuya colaboración basta para que un nodo acepte un valor como acordado. Un quórum contiene al menos un slice de cada uno de sus miembros.

Horizon Interfaz HTTP que expone las operaciones clásicas de Stellar (pagos, líneas de confianza, manejo de activos).

Soroban RPC Interfaz HTTP/JSON-RPC que expone simulación, envío y consulta de transacciones de contratos inteligentes.

Transacción y operación Una transacción es la unidad atómica firmada y enviada al ledger; puede contener hasta 100 operaciones, todas las cuales se aplican en un solo cierre de ledger o ninguna se aplica.

XLM (Lumen) Activo nativo de la red Stellar. Se utiliza para pagar las comisiones por transacción y, en este prototipo, como medio de pago de los boletos.

Stroop Unidad mínima de XLM. Un XLM equivale a 10^7 stroops. Se utiliza como unidad de almacenamiento por precisión entera; los costos en esta memoria se reportan convertidos a COP.

Base fee y surge pricing Tarifa base por operación ($\approx 0,02$ COP a la tasa de referencia) y mecanismo de precio dinámico que la red aplica en condiciones de saturación.

SAC (*Stellar Asset Contract*) Contrato que expone los activos de Stellar bajo la interfaz estándar de tokens de Soroban (**transfer**, **balance**, **approve**).

SEP-41 Estándar de interfaz para tokens en Soroban, compatible con el SAC.

Trustline (línea de confianza) Operación clásica de Stellar mediante la cual una cuenta declara su intención de recibir un activo emitido por otra cuenta.

Friendbot Servicio público de testnet que financia cuentas nuevas con XLM de prueba.

Ledger close time Intervalo entre cierres consecutivos del ledger; en Stellar es de aproximadamente 5 segundos.

Contratos inteligentes y NFT

Contrato inteligente Programa con estado persistente y ejecución determinística cuyas transiciones se validan en cada nodo bajo el mismo conjunto de reglas.

WASM (WebAssembly) Formato binario al que se compilan los contratos Soroban. El target utilizado en este trabajo es `wasm32v1-none`.

NFT (*Non-Fungible Token*) Token con identificador único, no intercambiable por otro del mismo tipo. En este trabajo cada boleto se representa como NFT.

Mint (acuñación) Operación de creación de un nuevo token. Equivale a la función `crear_ boleto` del contrato.

Burn (quema) Operación que retira un token del conjunto de tokens válidos. En el patrón *burn/remint*, la versión vigente se marca como inválida en cada reventa.

Burn/remint Patrón de transferencia de propiedad que combina la invalidación de la versión anterior con la emisión de una nueva versión a nombre del comprador, en una sola operación atómica.

Clawback Operación de Stellar Classic que permite al emisor revocar un activo previamente entregado, utilizada en este trabajo para la transferencia del coleccionable.

Atomicidad Propiedad por la cual una transacción o se ejecuta completamente o no se ejecuta en absoluto. Es la base de la compra de reventa, que combina pago, comisiones y cambio de propiedad.

Gas fee / Comisión Costo nominal asociado a la ejecución de una operación en la red. En Stellar es fijo ($\approx 0,02$ COP por operación a la tasa de referencia), a diferencia de redes basadas en *gas* variable.

Eventos (*events*) Mensajes que un contrato emite durante su ejecución, almacenados en el meta del ledger. Los consume el indexador para reconstruir el historial.

XDR (*External Data Representation*) Formato binario en el que Stellar serializa transacciones y respuestas. El *XDR proxy* construye transacciones en formato XDR sin firmar para que el comprador las firme localmente.

Componentes del prototipo

Freighter Extensión de navegador que actúa como billetera de Stellar. Permite al usuario firmar transacciones sin que la llave privada salga del entorno local.

Indexer (indexador) Proceso de larga duración que consulta el RPC y persiste los eventos del contrato en una base de datos relacional para consulta histórica.

XDR proxy stateless Patrón de backend que construye transacciones sin firmar y nunca custodia llaves privadas de usuarios finales.

Factory contract Contrato que despliega instancias independientes de otros contratos (en este trabajo, un `event_contract` por evento).

Verificador en puerta Rol autorizado por el organizador para invocar la operación de redención del boleto.

Redención Operación que marca un boleto como utilizado en el momento del ingreso al evento.

Estándares, normativa y siglas

SDLC (*Software Development Life Cycle*) Ciclo de vida del desarrollo de software.

KYC / AML *Know Your Customer* y *Anti-Money Laundering*. Procesos de identificación de usuarios y prevención de lavado de activos. Quedan fuera del alcance del prototipo.

Ley 1493 de 2011 Norma colombiana que regula los espectáculos públicos de las artes escénicas y establece una contribución parafiscal del 10 % sobre boletería que supera 3 UVT.

UVT (Unidad de Valor Tributario) Unidad de referencia de la tributación colombiana, actualizada anualmente.

API *Application Programming Interface*.

B2B *Business to Business*.

E2E *End to End*.

JWT *JSON Web Token*. Mecanismo de autenticación utilizado por el backend.

ORM *Object-Relational Mapping*. En el prototipo se utiliza Prisma sobre PostgreSQL.

RPC *Remote Procedure Call*.

SPA *Single Page Application*. Modelo del frontend.

TPS *Transactions per Second*.

1. INTRODUCCIÓN

La digitalización del mercado de boletería ha facilitado la compra y la transferencia de entradas, pero también ha incrementado los riesgos asociados a la reventa fraudulenta: duplicidad de códigos de acceso, venta concurrente del mismo activo a varios compradores y opacidad del historial de propiedad. Estos vectores afectan la confianza del comprador, la liquidación efectiva entre los actores del mercado y la trazabilidad operacional del organizador.

Este documento presenta el diseño, la implementación y la validación de **SecureTicket**, una solución B2B de reventa de entradas basada en contratos inteligentes Soroban sobre la red Stellar. La propuesta materializa cada boleto como un *token* no fungible (NFT) con versionado, aplica un patrón *burn/remint* en cada reventa para preservar la unicidad y ejecuta la liquidación de pagos y comisiones de forma atómica dentro de la misma invocación del contrato.

El documento se organiza en nueve secciones. La sección 2 presenta la descripción general del proyecto, incluyendo problema, propuesta de solución, objetivos y entregables. La sección 3 expone el contexto técnico y el análisis de soluciones comparables. La sección 4 detalla los requerimientos, restricciones y la especificación funcional del sistema. La sección 5 desarrolla la arquitectura y el diseño detallado de los contratos y los nodos de la solución. La sección 6 describe el proceso de implementación, despliegue e integración continua. La sección 7 consolida los resultados experimentales sobre la red de prueba (*testnet*). La sección 8 analiza el impacto del trabajo y presenta las conclusiones y el trabajo futuro. La sección 9 lista las referencias bibliográficas. Finalmente, los anexos referencian el Plan de Gestión del Proyecto, la Especificación de Requisitos de Software y el documento de Diseño y Propuesta del Trabajo de Grado, entregados como artefactos independientes.

2. DESCRIPCIÓN GENERAL Y PLANTEAMIENTO DEL PROBLEMA

2.1. Oportunidad, Problema

2.1.1. Contexto de la Industria: De lo Global a lo Local

La industria global del entretenimiento en vivo ha experimentado un crecimiento exponencial, consolidando la transición de la boletería física a la distribución electrónica. Sin embargo, este auge ha propiciado la proliferación de un mercado secundario (reventa) altamente especulativo y desregulado [1]. A nivel global, la alta demanda y la escasez artificial impulsan un mercado secundario de miles de millones de dólares anuales (*Nota: se recomienda insertar aquí la cita empírica global acordada*). En el contexto local latinoamericano y específicamente en Colombia, el ecosistema de conciertos y espectáculos públicos ha crecido de manera sostenida, regulado bajo normativas como la Ley 1493 de 2011, la cual establece contribuciones parafiscales sobre la boletería [2]. A pesar de los esfuerzos regulatorios, la informalidad en la reventa (operada principalmente a través de redes sociales y plataformas no oficiales) ha generado un ecosistema vulnerable, donde la asimetría de información perjudica sistemáticamente al consumidor final y priva a los organizadores de trazabilidad y recaudo sobre este mercado [1].

2.1.2. Formulación del Problema

El problema abordado por este trabajo se formula en los siguientes términos: *¿cómo diseñar un sistema B2B de reventa de entradas que garantice, por construcción, la unicidad y autenticidad de cada boleto, distribuya automáticamente las comisiones a organizador y plataforma, mantenga la trazabilidad pública del historial de propiedad y opere con tiempos de liquidación compatibles con la entrega inmediata del activo digital, sin requerir un intermediario centralizado de confianza?*

La formulación impone tres condiciones técnicas verificables: la unicidad debe ser una propiedad del registro, no del proceso de validación; la liquidación de pago y la transferencia de propiedad deben ocupar una sola unidad atómica; y el historial debe ser auditable por cualquier parte sin requerir privilegios concedidos por la plataforma.

2.1.3. Propuesta de Solución

La propuesta consiste en una plataforma B2B de reventa de entradas implementada como un prototipo distribuido en tres nodos (web, API y datos) que integra una capa de contratos inteligentes Soroban sobre la red Stellar. Cada boleto se modela como un NFT identificado por la tupla (`ticket_root_id`, `version`), donde el primer componente es estable durante toda la vida del activo y el segundo se incrementa en cada operación de reventa mediante un patrón *burn/remint*. La compra de reventa ejecuta en una sola transacción el pago al vendedor (92 %), la comisión al organizador (5 %), la comisión a la plataforma (3 %) y la transferencia de propiedad. La trazabilidad se materializa mediante siete eventos on-chain consumidos por un indexador que persiste el historial en una base de datos relacional.

El área de la propuesta es desarrollo de software con énfasis en sistemas distribuidos y aplicación de tecnologías de registro distribuido (*blockchain*).

2.1.4. Justificación de la Solución

La utilización de Stellar/Soroban se justifica por cuatro propiedades verificables en la documentación oficial de la red: tiempo de finalidad de 3 a 5 segundos por transacción, costo nominal fijo por operación de aproximadamente 0,02 COP¹, capacidad teórica del orden de 1 000 a 3 000 TPS, y semántica atómica de hasta 100 operaciones por transacción. Estas características hacen viable el diseño de una compra de reventa indivisible que combina pago, comisiones y propiedad, condición que no se logra de forma directa en esquemas tradicionales basados en pasarelas bancarias y registros centralizados.

Adicionalmente, el modelo de seguridad de la red, basado en el *Stellar Consensus Protocol* (SCP) bajo el paradigma *Federated Byzantine Agreement*, prioriza la propiedad de *safety* sobre *liveness*, lo cual es deseable en el caso de uso de reventa: ante un escenario adverso el ledger detiene su avance en lugar de producir bifurcaciones, lo que garantiza que ningún boleto pueda asignarse simultáneamente a dos compradores distintos en estados acordados como finales.

2.2. Descripción del Proyecto

2.2.1. Objetivo General

Desarrollar una solución B2B de reventa de tickets, basada en la tecnología de contratos inteligentes de Stellar, que garantice la unicidad y autenticidad de cada activo digital, reduciendo al mínimo el fraude y la especulación en la reventa de entradas para eventos de entretenimiento.

2.2.2. Objetivos Específicos

- Crear un sistema de reventa B2B de tickets basado en contratos inteligentes y NFTs que aseguren autenticidad, originalidad y trazabilidad.
- Desarrollar un contrato inteligente que genere tickets como NFTs, permita la transferencia de propiedad y automatice la distribución de comisiones.
- Implementar un prototipo que simule el proceso completo de reventa en la red de prueba de Stellar, desde la venta hasta la compra.
- Gestionar el funcionamiento de la herramienta a través de una implementación web que permita la simulación y visualización del modelo y de su integración con la cadena de bloques.

2.2.3. Entregables, Estándares y Justificación

La Tabla 1 consolida los entregables del trabajo de grado, los estándares aplicados como marco de referencia y la justificación de cada uno.

¹Equivalencia calculada a una tasa de referencia promediada de $1 \text{ XLM} \approx 1\,500 \text{ COP}$, debido a la alta variabilidad del precio del activo en el mercado. El costo on-chain real es de $100 \text{ stroops (base fee)}$; $1 \text{ XLM} = 10^7 \text{ stroops}$. Todas las conversiones a COP en esta memoria utilizan la misma tasa de referencia.

Tabla 1: Entregables y estándares aplicados.

Entregable	Estándar de referencia	Justificación
Memoria de Trabajo de Grado (este documento)	Formato institucional PUJ	Documento autocontenido que sintetiza el trabajo realizado.
Plan de Gestión del Proyecto (SPMP)	Plantilla institucional	Trazabilidad de la planeación, hitos, riesgos y presupuesto.
Especificación de Requisitos (SRS)	Plantilla institucional	Definición detallada de RF, RNF, casos de uso y criterios de aceptación.
Especificación del Diseño (SDD) y Propuesta de TG	Modelo C4 + secuencias UML	Vistas de contexto, contenedores, componentes y despliegue.
Suite de tres contratos Soroban + 58 pruebas	Rust + <code>soroban-sdk</code> 23	Materialización del modelo NFT con versionado, <i>burn/remint</i> y NFT SEP-41.
Backend Node.js + indexador	Express + Prisma + PostgreSQL	API REST, XDR proxy <i>stateless</i> y sincronización on-chain/off-chain.
Frontend web (React/Vite)	SPA responsive + Freightier	Flujo E2E para usuarios finales y operadores.
Integración continua	GitHub Actions	Compilación y pruebas automatizadas en cada <i>push</i> .

Los artefactos de gestión, requisitos, diseño y verificación se organizan de manera reproducible y auditable a partir del código fuente público y de la documentación entregada como anexos.

3. CONTEXTO DEL PROYECTO

3.1. Trasfondo

3.1.1. Introducción a la Tecnología Blockchain y Contratos Inteligentes

Una cadena de bloques (*blockchain*) se define formalmente como una máquina de estados replicada y de transición discreta, cuyo estado global $\mathcal{S}$ evoluciona mediante la aplicación sucesiva de transiciones de estado transaccionales agrupadas en bloques ordenados cronológicamente. Matemáticamente, el estado del sistema en el instante discreto t se formula como $\mathcal{S}_t = \Upsilon(\mathcal{S}_{t-1}, B_t)$, donde Υ representa la función de transición de estado global y B_t es el bloque validado e incorporado en el registro histórico. Cada bloque B_t constituye una estructura de datos inmutable enlazada criptográficamente con su predecesor inmediato a través de la inclusión del valor hash del encabezado anterior, denotado como $H(B_{t-1})$:

$$B_t = (H(B_{t-1}), \text{MerkleRoot}(T_t), \text{Timestamp}, \text{ConsensusMetadata}) \quad (1)$$

donde $H : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ es una función hash criptográfica unidireccional y resistente a colisiones (como la familia SHA-2 o SHA-3), y $\text{MerkleRoot}(T_t)$ es la raíz del árbol de Merkle asociado al conjunto de transacciones $T_t = \{t_1, t_2, \dots, t_n\}$ incorporadas en dicho bloque. Esta disposición de punteros criptográficos garantiza la inmutabilidad y la auditabilidad eficiente del sistema: cualquier alteración retrospectiva en una transacción elemental t_i se propaga recursivamente afectando la raíz de Merkle, los hashes intermedios y, en última instancia, el hash final del bloque correspondiente B_t , invalidando la consistencia de todos los bloques subsecuentes en la red distribuida.

Esta arquitectura de registro distribuido resuelve de manera determinística el problema clásico del doble gasto (*double-spending*) sin requerir una entidad central de compensación. En el ámbito de los activos digitales tradicionales, la duplicación ilimitada y sin costo de archivos impide el establecimiento directo de escasez digital. El doble gasto se presenta cuando una entidad maliciosa intenta transferir de forma simultánea un mismo recurso financiero digital a dos destinatarios distintos. La tecnología blockchain mitiga este vector de ataque imponiendo un orden cronológico y global inalterable de las transacciones a través de un protocolo de consenso descentralizado. De este modo, solo la primera transacción que cumple con las condiciones de validez sobre el estado actual $\mathcal{S}_{t-1}$ es procesada por los nodos de la red, mientras que cualquier transacción competidora subsiguiente que intente gastar el mismo activo es descartada determinísticamente al violar las reglas de balance y propiedad del estado resultante.

Históricamente, el concepto de cadena de bloques y su primera implementación funcional se remontan a la publicación del documento blanco firmado bajo el seudónimo de Satoshi Nakamoto en el año 2008, titulado «*Bitcoin: A Peer-to-Peer Electronic Cash System*» [1]. El surgimiento de esta tecnología responde a la necesidad estructural de prescindir de intermediarios centralizados de confianza (como bancos centrales, pasarelas de pago y cámaras de compensación) en las transacciones de valor en línea. Estos intermediarios tradicionales centralizan el control del flujo financiero, imponen estructuras tarifarias ineficientes y configuran puntos únicos de falla (*single points of failure*) vulnerables a la censura, ataques cibernéticos o inestabilidad sistémica.

Para ilustrar de forma clara e intuitiva el principio de inmutabilidad distribuida, se puede

plantear una analogía técnica basada en un libro contable físico compartido. Imagínese una red de pares en la que no existe un escribano central único que custodie las cuentas. En su lugar, cada miembro de la red posee una copia idéntica y sincronizada del libro. Cada vez que se desea registrar un conjunto de transacciones (una página del libro), se debe aplicar una regla de validación matemática uniforme evaluada de forma independiente por todos los participantes. Si un actor malicioso altera de forma unilateral una línea en una página de su copia local, la discrepancia matemática en la firma acumulada y en el hash de enlace con las páginas posteriores se hace evidente de inmediato. Al contrastar copias, el resto de los participantes identificará la incongruencia y rechazará la versión fraudulenta del registro, manteniendo la integridad del estado global a través de la redundancia y el consenso distribuido.

Sobre esta base, los contratos inteligentes (*smart contracts*), propuestos conceptualmente por Nick Szabo en 1997 [2] y popularizados con el despliegue de la red Ethereum en 2014 [3], representan programas con estado persistente y lógica determinística autoejecutable. Estos contratos se despliegan directamente en el registro criptográfico y son evaluados por todos los validadores de la red como parte de la función de transición Υ . Esto garantiza que sus reglas y compromisos se ejecuten exactamente según lo codificado, eliminando el riesgo de contraparte y la necesidad de supervisión manual externa.

3.1.2. La Red Stellar

Stellar es una red pública descentralizada orientada a la transferencia y movimiento eficiente de valor digital, a la tokenización de activos del mundo real (*Real World Assets* o RWA) y a la inclusión financiera a escala global. Co-fundada en el año 2014 por Jed McCaleb y Joyce Kim bajo la gestión de la Stellar Development Foundation (SDF), la plataforma fue concebida con el propósito de mitigar las fricciones existentes en los sistemas de pagos transfronterizos tradicionales, los cuales se caracterizan por una alta latencia de liquidación, elevados costos de corresponsalía bancaria y una marcada falta de interoperabilidad.

A nivel de arquitectura de red y especificación operativa, Stellar destaca por las siguientes características técnicas clave:

- **Eficiencia Energética y de Cómputo:** Al no depender de algoritmos de prueba de trabajo que exijan potencia de procesamiento masiva de carácter destructivo (como la minería ASIC de Bitcoin), Stellar opera mediante un protocolo de votación federada. El costo de procesamiento computacional por nodo es equivalente al de un servidor web estándar, reduciendo el consumo de energía y la huella de carbono del ledger global a valores insignificantes.
- **Baja Latencia de Cierre y Finalidad:** El tiempo de cierre de registro (*ledger close time*) promedio de Stellar oscila entre los 3 y 5 segundos. Esta característica provee una finalidad transaccional absoluta e inmediata (*immediate finality*), eliminando la necesidad de esperar confirmaciones heurísticas probabilísticas tras varias horas de computación, lo que resulta fundamental para procesos de emisión y validación de boletería digital en tiempo real.
- **Costos Operativos Insignificantes:** El costo base por operación en la red está configurado nominalmente en 100 stroops (donde $1 \text{ XLM} = 10^7 \text{ stroops}$), lo cual equivale

aproximadamente a 0,02 COP a la tasa de referencia histórica de conversión. Esta tarifa nominal actúa como un desincentivo criptoeconómico contra ataques de denegación de servicio por saturación de transacciones (DoS), permitiendo a la vez un volumen masivo de operaciones comerciales y de reventa sin penalizar económicamente a los usuarios del ecosistema.

3.1.3. Protocolo de Consenso Stellar (SCP) y Acuerdo Bizantino Federado (FBA)

El mecanismo de acuerdo y validación de la red Stellar está instrumentado bajo el Protocolo de Consenso Stellar (SCP, *Stellar Consensus Protocol*) diseñado por David Mazières en 2015 [4]. El SCP constituye la primera formulación académica y comercialmente implementada del paradigma del Acuerdo Bizantino Federado (FBA, *Federated Byzantine Agreement*).

Los sistemas clásicos de consenso tolerantes a fallas bizantinas (BFT), como el célebre protocolo PBFT (*Practical Byzantine Fault Tolerance*) [5], imponen un modelo de membresía estática y cerrada. En estos esquemas, la participación requiere que una autoridad centralizada o un consorcio defina formalmente un conjunto estático de validadores $\mathbf{R}$. Todos los nodos de la red deben conocer y concordar unívocamente sobre la membresía del sistema en todo instante. Este enfoque limita drásticamente la descentralización al restringir el acceso libre de nodos e induce una penalización severa en el rendimiento debido a la complejidad de mensajería bidireccional exhaustiva del orden de $\mathcal{O}(|\mathbf{R}|^2)$ en su fase de confirmación.

En contraste, el FBA rompe con el supuesto de membresía cerrada y estática. En una red FBA, no existe una lista centralizada de validadores aprobados; cualquier nodo puede unirse al sistema y participar en el proceso de consenso de manera orgánica. Para alcanzar el acuerdo sin un listado global, el FBA introduce los siguientes conceptos matemáticos fundamentales:

- **Rebanada de Quórum (*Quorum Slice*):** Para un nodo individual $v \in V$, donde V representa el conjunto de todos los nodos de la red, una rebanada de quórum $S(v)$ es un subconjunto no vacío de nodos tal que $v \in S(v) \subseteq V$, en el cual v deposita su confianza de forma soberana e independiente para ratificar una transacción o transición de estado. Cada nodo puede definir de forma autónoma una o múltiples rebanadas de quórum alternativas (por ejemplo, basadas en organizaciones independientes o nodos altamente reputados) para estructurar su topología de confianza.
- **Quórum:** Un subconjunto de nodos $U \subseteq V$ constituye un quórum si es no vacío y para cada miembro del subconjunto $u \in U$, existe al menos una rebanada de quórum $S(u)$ tal que $S(u) \subseteq U$. De esta forma, las decisiones de aceptación de estado no se determinan por una mayoría absoluta y global, sino de manera inductiva a través de la intersección local y superpuesta de las rebanadas de quórum declaradas por cada participante.

Bajo el marco del Teorema CAP, que establece los límites fundamentales de consistencia (*Safety*), disponibilidad (*Liveness*) y tolerancia a la partición de red en sistemas distribuidos, el SCP prioriza strictly la consistencia sobre la disponibilidad. En la práctica, ante un fallo catastrófico de conectividad, un ataque bizantino masivo o una configuración errónea de confianza que impida la convergencia y la intersección de quóruns, el protocolo prefiere congelar el avance de la red y suspender la confirmación de transacciones (*halt*) antes que permitir la generación de una bifurcación histórica en el registro. Esta priorización es la única coherente

con sistemas de transferencia de activos financieros y boletería digital: la suspensión temporal del servicio resulta preferible a la confirmación de transacciones divergentes que posibiliten que una misma entrada digital sea asignada de manera definitiva a múltiples compradores concurrentes.

A nivel energético y de gobernanza, el SCP exhibe una eficiencia superior en comparación con los esquemas de consenso preponderantes en la industria:

- **Proof of Work (PoW):** El consenso se alcanza mediante una carrera computacional competitiva de fuerza bruta para encontrar el hash de un bloque que cumpla con una dificultad dinámica. Esto implica un desperdicio energético masivo, del orden de teravatios-hora a nivel global, y restringe el rendimiento a unas pocas transacciones por segundo debido a los tiempos de dificultad adaptativa.
- **Proof of Stake (PoS):** Si bien mitiga el consumo de energía, la votación está directamente correlacionada con el capital criptográfico depositado o bloqueado (*staking*). Este modelo introduce riesgos de monopolio y consolidación plutocrática del control de la red por parte de los tenedores mayoritarios de tokens, además de requerir algoritmos complejos para resolver escenarios de bifurcación sin costo (*nothing-at-stake*) y mecanismos de penalización monetaria extrema (*slashing*).
- **Stellar Consensus Protocol (SCP):** El acuerdo se construye puramente sobre el intercambio de mensajes criptográficamente firmados entre nodos validados según la topología descentralizada de las rebanadas de confianza. Al no requerir computación intensiva de hashing ni bloqueos financieros coercitivos de capital para la ponderación del voto, el consumo energético de la red Stellar se asemeja al de la infraestructura de mensajería de un sistema web tradicional de alta eficiencia, asegurando rapidez y un costo de carbono mínimo.

3.1.4. Tokens Estables (Stablecoins) y Bases Matemáticas del Consenso

Las monedas estables (*stablecoins*) son representaciones digitales de valor emitidas sobre un registro blockchain cuyo valor de intercambio comercial se encuentra referenciado o respaldado en paridad unitaria (1 : 1) con respecto a un activo tradicional de baja volatilidad, comúnmente una moneda fiduciaria de curso legal como el dólar estadounidense (USD). En la arquitectura Web2.5 planteada en este trabajo, el empleo de tokens estables como USDC (desplegado nativamente en Stellar mediante el *Stellar Asset Contract* o SAC) resulta un pilar conceptual fundamental para solventar la alta variabilidad del activo nativo de la red (XLM). La volatilidad inherente de los criptoactivos nativos impediría la adopción del sistema por parte de operadores logísticos y usuarios finales, debido al riesgo cambiario implícito en los intervalos temporales de reventa. El uso de un token digital convertible a paridad fiduciaria asegura la certidumbre económica del precio final y de las comisiones del mercado secundario.

La estabilidad y consistencia global del Acuerdo Bizantino Federado descansa sobre la base matemática de la **intersección de quóruns**. Decimos formalmente que un sistema de quóruns $\mathcal{Q}$ para una red federada V posee la propiedad de intersección de quóruns si para todo par de quóruns $U_1, U_2 \in \mathcal{Q}$, se cumple que:

$$U_1 \cap U_2 \neq \emptyset \tag{2}$$

y dicha intersección contiene al menos un nodo correcto (no defectuoso ni bizantino). Esta propiedad garantiza matemáticamente que los nodos honestos de la red no podrán validar transacciones contradictorias en registros paralelos, dado que cualquier decisión final de consenso en un quórum requiere la ratificación de nodos compartidos con el otro quórum. La conformación adecuada de las rebanadas de quórum por parte de los operadores de nodos asegura que la red mantenga la intersección de quóruns de forma dinámica, tolerando hasta un tercio ($f < n/3$) de fallos bizantinos locales en las subestructuras de confianza sin comprometer la consistencia global del ledger distribuido.

3.1.5. Soroban, NFTs y tokens de pago

Soroban es la plataforma de contratos inteligentes de Stellar. Los contratos se escriben en Rust contra el crate `soroban-sdk` (versión 23 en este trabajo) y se compilan al target `wasm32v1-none`. El modelo de ejecución es determinístico, sin acceso a fuentes no reproducibles (relojes locales, aleatoriedad no derivada del ledger, llamadas al sistema operativo).

Soroban gestiona tres tipos de almacenamiento: `Persistent`, `Temporary` e `Instance`, cada uno con un costo de *rent* distinto. La emisión de eventos se realiza mediante `events().publish()`: cada evento contiene *topics* indexables y un *body* serializable, y se almacena en el meta del ledger, lo que permite a los indexadores externos consumirlo sin afectar el costo de *rent* del contrato.

El estándar NFT en Soroban define una representación de activos no fungibles con identificador único, propietario tipo `Address` y reglas de transferencia controladas por el contrato emisor. En este trabajo se extiende este estándar con un esquema de versionado: cada boleto se identifica por la tupla `(ticket_root_id, version)`, donde `version` se incrementa en cada operación de reventa mediante un patrón *burn/remint*.

El activo nativo de la red, XLM, y cualquier activo emitido en Stellar se exponen como tokens compatibles con la interfaz estándar de Soroban a través del *Stellar Asset Contract* (SAC). Esto permite que las operaciones de pago de la compra de reventa se ejecuten dentro del mismo flujo de invocación del contrato del evento, sin transacciones separadas para el pago y la transferencia de propiedad.

3.2. Análisis del Contexto

El análisis comparativo de soluciones del mercado y de la literatura técnica permite identificar tres categorías de antecedentes: (i) plataformas tradicionales de reventa basadas en QR estáticos y conciliación bancaria; (ii) soluciones NFT desplegadas sobre Ethereum 1.0 o sus capas 2; y (iii) propuestas académicas de *ticketing* distribuido sin liquidación atómica. La Tabla 2 contrasta los principales parámetros técnicos de cada categoría contra la propuesta de este trabajo.

Tabla 2: Análisis comparativo de soluciones del contexto.

Parámetro	Tradicional	NFT en ETH 1.0	Académica	Propuesta
Finalidad	72–120 h	1–6 min	Variable	3–5 s
Costo por op.	1,5–3,5 % + fija	USD 5–50 (gas)	N/A	$\approx 0,02$ COP
Capacidad	—	15–30 TPS	—	1 000–3 000 TPS
Atomicidad pago + propiedad	No	Parcial	No	Sí
Trazabilidad pública	No	Sí	Parcial	Sí
Unicidad por construcción	No	Sí	Variable	Sí (<i>burn/remint</i>)

La principal ventaja de la propuesta frente al esquema NFT clásico sobre Ethereum es la liquidación atómica de pago, comisiones y propiedad dentro de una sola transacción, sin requerir patrones de *commit-reveal* o *escrow* externos. Frente a las plataformas tradicionales, la unicidad del NFT con versionado y el patrón *burn/remint* eliminan por construcción la coexistencia de dos copias válidas del mismo activo, lo cual no es alcanzable mediante validación posterior de QR estáticos.

Como referencia normativa local, la Ley 1493 de 2011 regula los espectáculos públicos de las artes escénicas en Colombia y establece una contribución parafiscal del 10 % sobre la boletería cuyo precio individual sea igual o superior a tres UVT. El modelo propuesto registra el precio efectivo on-chain en cada operación, lo que habilita el cálculo determinístico de la contribución mediante una agregación sobre el historial indexado. Este mapeo se utiliza como referencia académica y no constituye declaración de cumplimiento regulatorio productivo.

4. ANÁLISIS DEL PROBLEMA

4.1. Requerimientos

El sistema se especifica mediante veinte requerimientos funcionales (RF) y veinte requerimientos no funcionales (RNF), organizados en cinco módulos: M1 gestión de eventos y tickets, M2 emisión NFT, M3 mercado secundario, M4 configuración y políticas, M5 usuarios y autenticación. La especificación detallada con criterios de aceptación y trazabilidad a casos de uso se entrega como anexo (SRS). Las Tablas 3 y 4 resumen el conjunto completo.

Tabla 3: Requerimientos funcionales del sistema (resumen).

Id.	Descripción	Módulo
RF-01	Comprar boletos para un evento mediante interfaz clara.	M1, M5
RF-02	Validar que la compra sea autorizada (reglas, permisos, control de usuario).	M1, M4, M5
RF-03	Generar identificador único para cada boleto, vinculable al NFT.	M1, M2
RF-04	Permitir compra con flujo sencillo.	M1, M5
RF-05	Autenticar al usuario antes de comprar.	M5
RF-06	Registrar propietario inicial y final del boleto (trazabilidad).	M1, M2, M3, M5
RF-07	Listar boleto para reventa.	M3, M4, M5
RF-08	Verificar que el usuario sea dueño antes de revender.	M2, M3, M5
RF-09	Ejecutar compra en reventa de forma atómica.	M2, M3, M4, M5
RF-10	Evitar que el boleto se venda más de una vez.	M1, M2, M3
RF-11	Ejecutar proceso de <i>burn</i> del boleto vendido.	M2, M3
RF-12	Generar nuevo token para nuevo propietario.	M2, M3, M5
RF-13	Mantener historial de transferencias.	M1, M2, M3, M5
RF-14	Consultar estado de boletos adquiridos.	M1, M2, M5
RF-15	Verificación rápida del boleto.	M1, M2
RF-16	Solo el propietario puede modificar el precio de reventa.	M3, M4, M5
RF-17	Integrarse con contratos inteligentes Soroban.	M2, M3, M4
RF-18	Modificar precio del boleto listado para reventa.	M3, M4, M5
RF-19	Consultar disponibilidad de boletos.	M1
RF-20	Registrar cada transacción realizada.	M1, M2, M3, M5

Los RNF cubren desempeño, seguridad, usabilidad, disponibilidad, mantenibilidad e integración.

Tabla 4: Requerimientos no funcionales del sistema (resumen).

Id.	Descripción
RNF-01	Procesar la compra de boletos en menos de 5 segundos.
RNF-02	Soportar al menos 100 usuarios concurrentes.
RNF-03	Tener disponibilidad mínima del 99,5 % mensual.
RNF-04	Bloquear transacciones tras 3 intentos fallidos consecutivos.
RNF-05	Cifrar comunicaciones mediante TLS 1.2 o superior.
RNF-06	Soportar al menos 2 400 transacciones diarias.
RNF-07	Completar el proceso de compra en máximo 5 pasos.
RNF-08	Permitir aprendizaje del usuario en menos de 15 minutos.
RNF-09	Mantener una tasa de fallos menor al 0,5 %.
RNF-10	Garantizar atomicidad en el 100 % de las transacciones.
RNF-11	Tener cobertura de pruebas mínima del 70 %.
RNF-12	Ser compatible con navegadores web modernos.
RNF-13	Integrarse con contratos Soroban mediante API REST.
RNF-14	Almacenar <i>logs</i> por mínimo 12 meses.
RNF-15	Cerrar sesión tras 30 minutos de inactividad.
RNF-16	No superar 5 horas de inactividad mensual.
RNF-17	Verificar boletos en menos de 5 segundos.
RNF-18	Soportar varios eventos simultáneos.
RNF-19	Recuperarse de fallos en menos de 20 minutos.
RNF-20	Conservar historial de boletos por mínimo 5 años.

4.2. Restricciones

Las restricciones del prototipo se agrupan en cuatro categorías:

- **De alcance.** El sistema se limita al mercado secundario de entradas para eventos de entretenimiento. Quedan fuera la integración con pasarelas de pago fiat reales, los procesos KYC, el despliegue en *mainnet* y el acoplamiento con plataformas comerciales de *ticketing*.
- **Técnicas.** El despliegue se realiza sobre la red de prueba (*testnet*) de Stellar. La compra del comprador final se firma exclusivamente desde la extensión *Freighter* en el navegador. Las operaciones clásicas (**change_trust**, **payment**) se enrutan por *Horizon*; las invocaciones de contrato, por *Soroban RPC*. El indexador no se ejecuta en entornos *serverless* dado que requiere proceso de fondo persistente.
- **Normativas.** La Ley 1493 de 2011 se incluye como referencia académica para la fiscalización digital de espectáculos públicos en Colombia.
- **Operativas.** La política de comisiones del prototipo se fija en 5 % organizador y 3 % plataforma; estos parámetros son configurables por contrato pero no se modifican durante la evaluación. La ventana de retención del método **getEvents** del RPC público obliga al indexador a poll-ear cada 5 segundos.

4.3. Especificación Funcional

El sistema involucra cuatro actores principales, alineados con los casos de uso del SDD. Cada rol queda acotado y, cuando interviene on-chain, queda verificado por el contrato mediante **require_auth()** sobre la dirección Stellar correspondiente:

- **Administrador de la Plataforma.** Configura eventos en el panel administrativo y dispara el despliegue de contratos de evento mediante el `factory_contract`.
- **Organizador del evento (ticketera B2B).** Única dirección autorizada para invocar `crear_boleto`, registrar verificadores e invalidar boletos en un contrato de evento. Es receptor de las comisiones de organizador.
- **Usuario.** Comprador y vendedor del mercado primario y secundario. El propietario actual firma `listar_boleto` y `cancelar_venta`; el comprador firma `comprar_boleto` desde Freighter.
- **Personal del Evento (verificador en puerta).** Rol STAFF o ADMIN registrado por el organizador. Ejecuta la redención en puerta a través del endpoint `POST /api/admin/scan`, con respaldo on-chain mediante `redimir_boleto`.

El diagrama de casos de uso (Figura 1) sintetiza la interacción de los cuatro actores con las funcionalidades principales del sistema, agrupadas por módulo. La especificación detallada de cada caso de uso (CU-01 Crear evento y desplegar contrato, CU-02 Compra primaria, CU-03 Listar para reventa, CU-04 Compra en reventa atómica, CU-05 Verificar en puerta, CU-06 Invalidar boleto) se documenta en el anexo SDD.

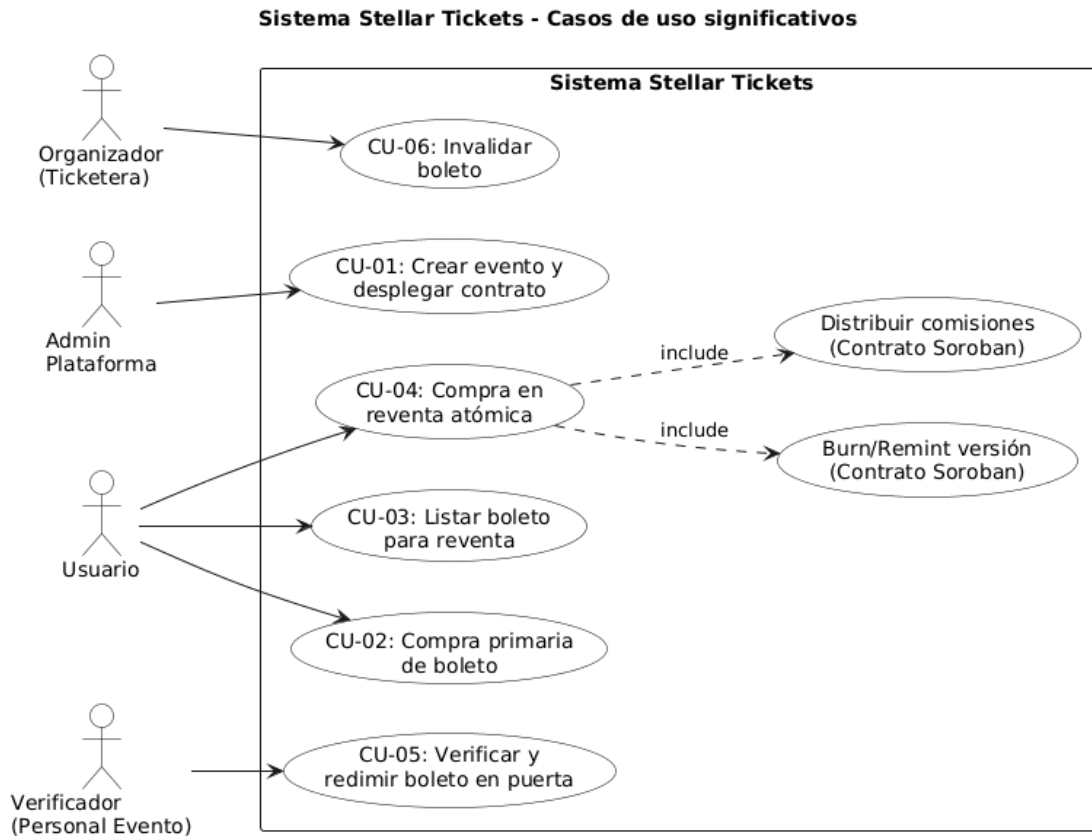


Figura 1: Diagrama de casos de uso del sistema (4 actores, 6 casos de uso principales).

El ciclo de vida funcional del boleto comprende seis fases observables on-chain: *creación* por parte del organizador; *listado* en mercado secundario; *compra* (primaria o reventa); *transferencia de propiedad* mediante *burn/remint*; *redención* por un verificador autorizado; y *cancelación administrativa* por el organizador en casos excepcionales. Las consultas de solo lectura cubren la verificación de propiedad, la versión vigente, el conjunto de boletos en reventa y el historial completo del evento.

La especificación detallada de casos de uso, los criterios de aceptación por operación y la trazabilidad entre RF, operaciones del contrato y pruebas unitarias se documentan en el anexo SRS.

5. DISEÑO DE LA SOLUCIÓN

Esta sección presenta la arquitectura del sistema y los artefactos del diseño detallado. La justificación de las tecnologías seleccionadas frente a alternativas se documenta en el anexo de Diseño y Propuesta del Trabajo de Grado.

El diagrama de contexto (Figura 2) ubica al sistema dentro de su ecosistema: usuarios finales y organizadores interactúan con SecureTicket, que a su vez se comunica con la red Stellar (Soroban RPC y Horizon) y con la billetera Freightier como agente firmante del lado del usuario.

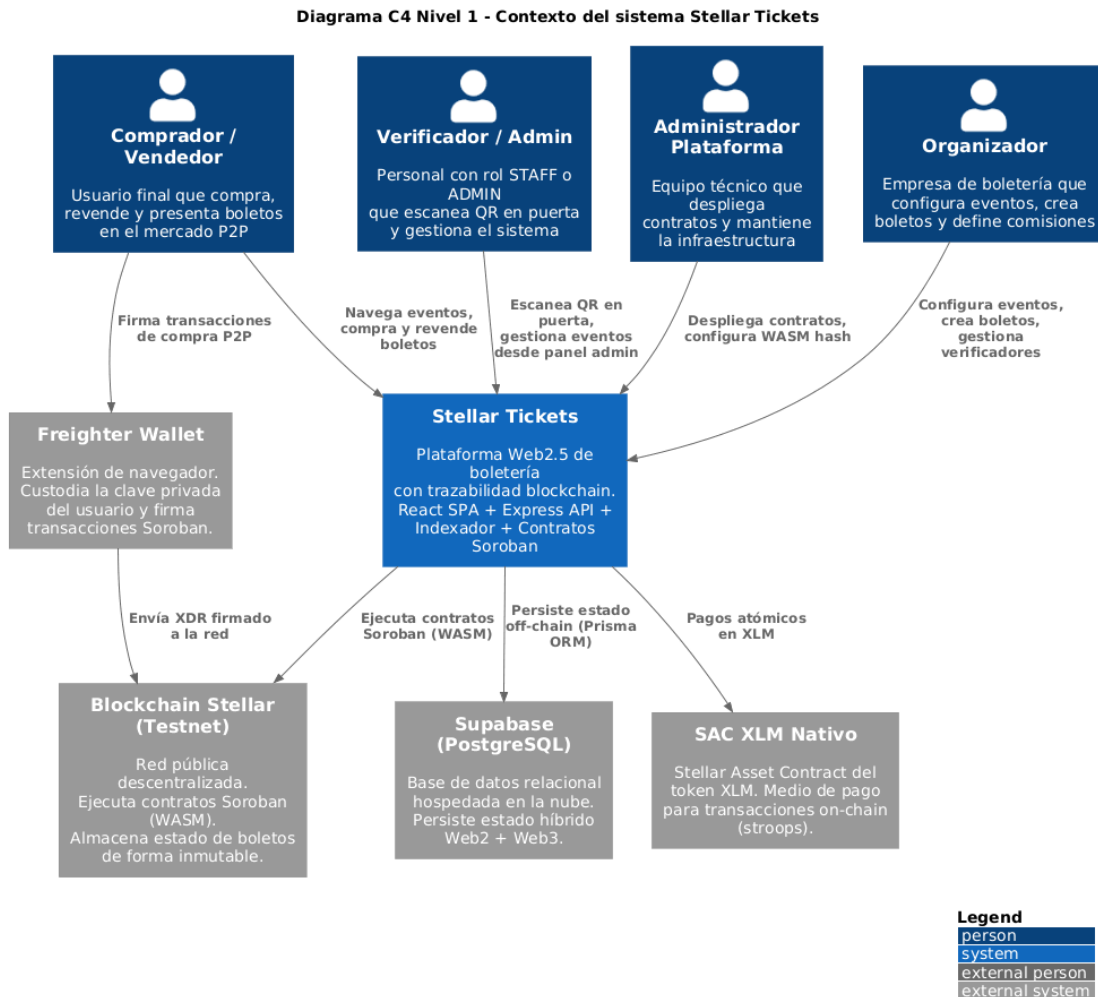


Figura 2: Diagrama de contexto C4 (nivel 1) del sistema SecureTicket.

5.1. Actores, roles y límites de confianza

El sistema separa dos planos de confianza. El plano *on-chain* reúne a los actores que firman operaciones del contrato (organizador, propietario, comprador, verificador, plataforma, administrador del *factory*); todas sus interacciones son validadas por el SCP y registradas en el

ledger. El plano *off-chain* reúne al backend, al indexador y a la base de datos relacional. El backend opera bajo un modelo *XDR proxy stateless*: construye transacciones sin firmar cuando el firmante es el comprador, y firma con la llave del organizador únicamente las operaciones que corresponden a su rol. En ningún momento custodia llaves privadas de usuarios finales.

5.2. Modelo del ticket NFT

Cada boleto se modela en `event_contract` mediante la estructura `Boleto`, con campos `ticket_root_id` (u32, estable), `version` (u32, incrementable en reventa), `id_evento`, `propietario` (`Address`), `precio` (i128, almacenado en la unidad mínima de XLM por precisión entera), `en_venta`, `es_reventa`, `usado` e `invalidado`.

La unicidad se garantiza por la clave de almacenamiento `Boleto(ticket_root_id, version)` y por el contador `ContadorBoletos` que asigna identificadores monotónicamente crecientes. La autenticidad se valida mediante la firma del organizador al emitir y por la firma del propietario en cada operación posterior. La trazabilidad se materializa mediante eventos on-chain persistidos por el indexador.

5.2.1. Identificador y separación de datos sensibles

El contrato almacena exclusivamente datos operacionales públicos (identificadores, propietario, precio, estados booleanos). Los datos personales del comprador se almacenan off-chain bajo el esquema `ticketing` de PostgreSQL, asociados por `user_id` y nunca por la llave privada. La vinculación on-chain/off-chain se realiza por la dirección Stellar (`wallet_address`) que el usuario asocia voluntariamente al conectar Freightier.

5.2.2. Estados y patrón burn/remint

El ciclo de vida se modela mediante tres estados: `ACTIVE` (`usado=false`, `invalidado=false`), `USED` (`redimir_boleto` por verificador) y `CANCELLED` (`invalidar_boleto` por organizador). En cada reventa se aplica el patrón *burn/remint*: la versión vigente se marca como `invalidado=true` y se inserta una nueva entrada `Boleto(ticket_root_id, version+1)` con el comprador como propietario. El índice `VersionActual(ticket_root_id)` se actualiza para que las consultas apunten a la nueva versión. La Figura 3 ilustra las transiciones permitidas.

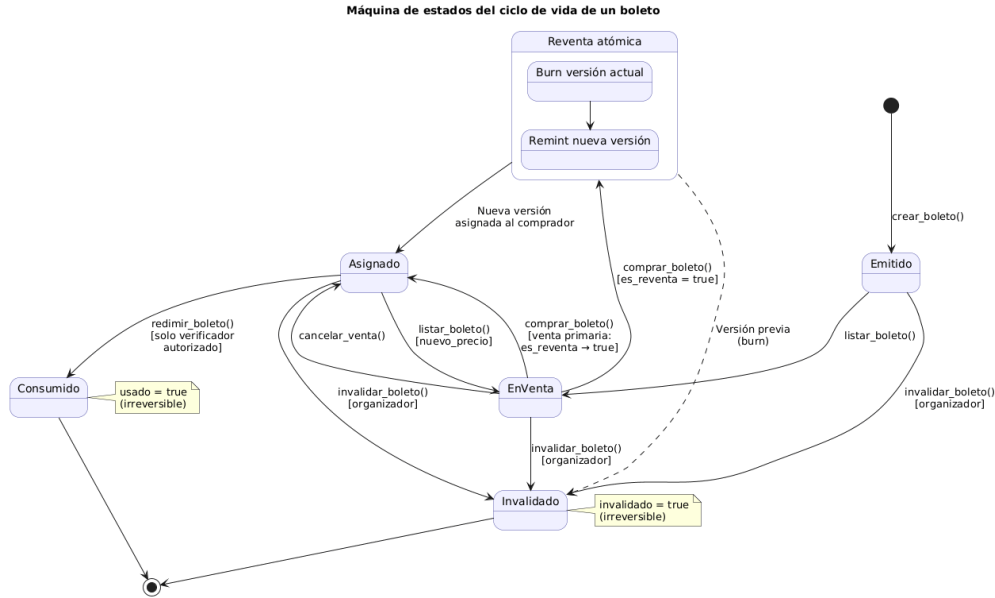


Figura 3: Diagrama de estados del boleto: ACTIVE, USED y CANCELLED, con sus transiciones.

5.3. Diseño de contratos inteligentes

El sistema utiliza tres contratos Soroban con responsabilidades acotadas: `factory_contract`, `event_contract` y `ticket_nft_contract`. Esta separación responde al principio de aislamiento de fallos: el estado de cada evento queda confinado a su propio par de contratos y la lógica de emisión NFT se desacopla de las reglas comerciales.

5.3.1. `factory_contract`

Implementa el patrón *Factory* para el despliegue de un `event_contract` independiente por cada evento. Su estructura `ConfiguracionEvento` agrupa `id_evento`, `organizador`, `token_pago`, `comision_organizador`, `comision_plataforma`, `wallet_organizador`, `wallet_plataforma` y `capacidad_total`. La función `crear_evento_contrato` requiere doble autenticación (administrador del *factory* y organizador) y emite el evento `EventoCreado`. El despliegue utiliza `deployer().with_current_contract(salt).deploy_v2(hash, ())`.

5.3.2. `event_contract`

Expone el conjunto funcional con identificadores en español por trazabilidad académica: `inicializar`, `crear_boleto`, `listar_boleto`, `cancelar_venta`, `comprar_boleto`, `agregar_verificador`, `remover_verificador`, `redimir_boleto`, `invalidar_boleto` y un conjunto de consultas de solo lectura.

La operación `comprar_boleto` aplica dos flujos según el campo `es_reventa`:

- **Venta primaria** (`es_reventa=false`): el 100 % del precio se transfiere al organizador y se marca `es_reventa=true`.

- **Reventa** (`es_reventa=true`): el contrato calcula tres montos sobre el precio listado (`comision_organizador` 5 %, `comision_plataforma` 3 %, `monto_vendedor` 92 %), ejecuta las tres transferencias y el cambio de propiedad en la misma invocación.

La constante `BASE_PORCENTAJE` se fija en 100 (`i128`) para aritmética entera exacta. Las comisiones se validan en `inicializar` para que su suma no supere el 100 %.

5.3.3. `ticket_nft_contract`

Implementación SEP-41-compatible para la representación NFT del boleto en un contrato dedicado (aproximadamente 12 KB de WASM, 10 pruebas unitarias). Expone las operaciones `mint(propietario, ticket_root_id, metadata_url)`, `transfer(from, to, token_id)` y `burn(token_id)`. Cada evento despliega su propio `ticket_nft_contract`, en correspondencia uno a uno con su `event_contract`. El `nft_token_id` se renueva en cada reventa para que las URLs cacheadas en Freightier del propietario anterior no sirvan como prueba válida en puerta.

5.3.4. Errores tipados y eventos on-chain

El enum `ErrorContrato` define dieciséis códigos: 1 `YaInicializado`, 2 `ComisionesMuyAltas`, 3 `ComisionesNegativas`, 4 `PrecioInvalido`, 5 `YaEnVenta`, 6 `BoletoUsado`, 7 `NoEnVenta`, 8 `AutoCompra`, 9 `YaUsado`, 10 `BoletoNoEncontrado`, 11 `NoAutorizado`, 12 `VersionInvalida`, 13 `BoletoInvalidado`, 14 `NoInicializado`, 15 `VerificadorYaExiste`, 16 `VerificadorNoEncontrado`. Los siete eventos emitidos son `boleto_creado`, `boleto_listado`, `venta_cancelada`, `boleto_comprado_primario`, `boleto_revendido`, `boleto_redimido` y `boleto_invalidado_evt`.

5.4. Arquitectura distribuida (tres nodos)

El prototipo se despliega en tres nodos lógicos con responsabilidades diferenciadas, lo que permite escalar de forma independiente cada plano y aislar fallos por dominio. La Figura 4 muestra el diagrama C4 de contenedores con los flujos de datos entre los nodos web, API y datos, y su conexión con la red Stellar.

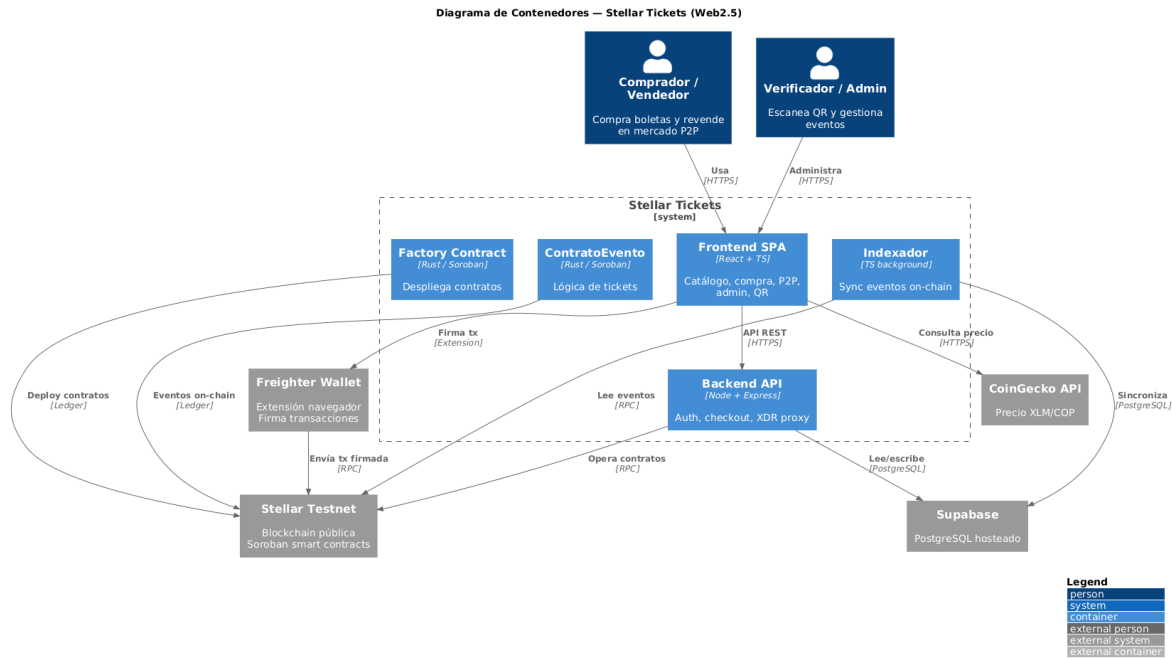


Figura 4: Diagrama C4 de contenedores (nivel 2): nodos web, API y datos, y su acoplamiento con Soroban RPC y Horizon.

5.4.1. Nodo web (frontend)

Aplicación React 18 con Vite, TypeScript, Tailwind CSS y la familia de componentes shadcn/ui (sobre Radix). La integración Web3 utiliza `@stellar/freighter-api` 6.0.1 para la firma del comprador y `@stellar/stellar-sdk` 14.6.1 para la construcción local de transacciones de validación. Expone 23 rutas que cubren el flujo de e-commerce (catálogo, carrito, *checkout*, mis entradas, mis ventas P2P) y los flujos administrativos (panel y escáner QR).

5.4.2. Nodo API (backend)

Backend Node.js con Express y TypeScript, sobre Prisma como ORM contra PostgreSQL. El módulo de autenticación utiliza `bcryptjs` (factor 10) y `jsonwebtoken` (vigencia 7 días). La integración con Stellar utiliza `@stellar/stellar-sdk` 14.4.1 (versión fijada). Expone tres familias de endpoints: catálogo público con caché en memoria (TTL 30–60 s), autenticación y *checkout* fiat transaccional, e integración Soroban con los endpoints `secure-ticket`, `list-ticket`, `cancel-listing`, `build-buy-xdr`, `submit` y la cadena Stellar Classic.

5.4.3. Nodo datos (PostgreSQL + indexador)

PostgreSQL hospedado en Supabase, esquema `ticketing`, `connection_limit=5`. El esquema comprende los modelos del dominio Web2 (`users`, `events`, `organizers`, `venues`, `event_ticket_types`, `event_seat_inventory`, `carts`, `orders`, `payments`, `tickets`) y los campos Web3 añadidos a `tickets`: `contract_address`, `ticket_root_id`, `version`, `is_for_sale`, `resale_price` (BigInt, unidad mínima de XLM), `owner_wallet` y `asset_code`. La unicidad

on-chain se refleja en el índice único (`contract_address`, `ticket_root_id`, `version`).

El indexador es un proceso de larga duración que consulta el RPC cada 5 segundos. Mantiene un cursor en `indexer_state.last_ledger`, procesa los siete eventos del contrato con semántica idempotente y reposiciona el cursor automáticamente si cae fuera de la ventana de retención del RPC público.

5.5. Coleccionable Stellar Classic

Como capa visual adicional, el sistema emite un activo Stellar Classic asociado a cada boleto (código T + 11 hex derivados del UUID). El emisor es la dirección del organizador con flags `AUTH_REVOCABLE` y `AUTH_CLAWBACK_ENABLED`. El flujo opera mediante cuatro endpoints: `build-trust-xdr`, `submit-classic`, `mint-collectible` (`PAYMENT` idempotente) y `transfer-collectible` (`CLAWBACK` + `PAYMENT`). Esta capa no sustituye al modelo Soroban como fuente de verdad.

6. DESARROLLO DE LA SOLUCIÓN

Esta sección describe el proceso seguido para materializar el diseño en un prototipo operativo sobre *testnet*. La metodología combinó cuatro fases articuladas: revisión técnica, diseño de arquitectura, implementación incremental y evaluación experimental, ejecutadas bajo un esquema ágil con tablero Kanban y revisiones semanales con el director del trabajo de grado. El detalle de la planeación se documenta en el anexo SPMP.

6.1. Implementación de contratos Soroban

Los contratos se desarrollan en Rust contra `soroban-sdk` versión 23, organizados como un Cargo *workspace* con tres paquetes: `event_contract`, `factory_contract` y `ticket_nft_contract`. El perfil de compilación `release` aplica `opt-level="z"`, *Link-Time Optimization*, eliminación de símbolos y `panic="abort"`, lo que produce módulos WASM compatibles con el target `wasm32v1-none`.

El contrato `event_contract` comprende aproximadamente 1142 líneas de código fuente en `lib.rs` y 709 líneas de pruebas en `test.rs`, con 35 casos unitarios. El contrato `factory_contract` comprende 431 líneas de código y 245 de pruebas, con 13 casos. El contrato `ticket_nft_contract` (introducido en la fase 4.5 del proyecto, estilo SEP-41) ocupa aproximadamente 12 KB de WASM y agrega 10 casos. El total de pruebas automatizadas es de 58 casos, ejecutables mediante `cargo test` en el *workspace*.

6.1.1. NFT del boleto: emisión, propiedad y versionado

El boleto se modela bajo la estructura `Boleto` y la clave de almacenamiento `ClaveDato::Boleto(u32, u32)`, donde la tupla corresponde a `(ticket_root_id, version)`. La asignación del `ticket_root_id` se realiza en `crear_boleto` mediante el contador `ContadorBoletos`, que se incrementa atómicamente por cada emisión. Cada cambio de propiedad en reventa incrementa `version` y actualiza el puntero `VersionActual(ticket_root_id)`.

6.1.2. Pagos atómicos vía SAC

El activo de pago utilizado es XLM expuesto vía *Stellar Asset Contract* (SAC). La transferencia se realiza mediante invocación al método `transfer` del SAC dentro del mismo *call frame* de `comprar_boleto`, lo que garantiza que el pago, la distribución de comisiones y el cambio de propiedad ocupen una única transacción del ledger. La Figura 5 muestra el diagrama de secuencia del flujo crítico de compra en reventa atómica.

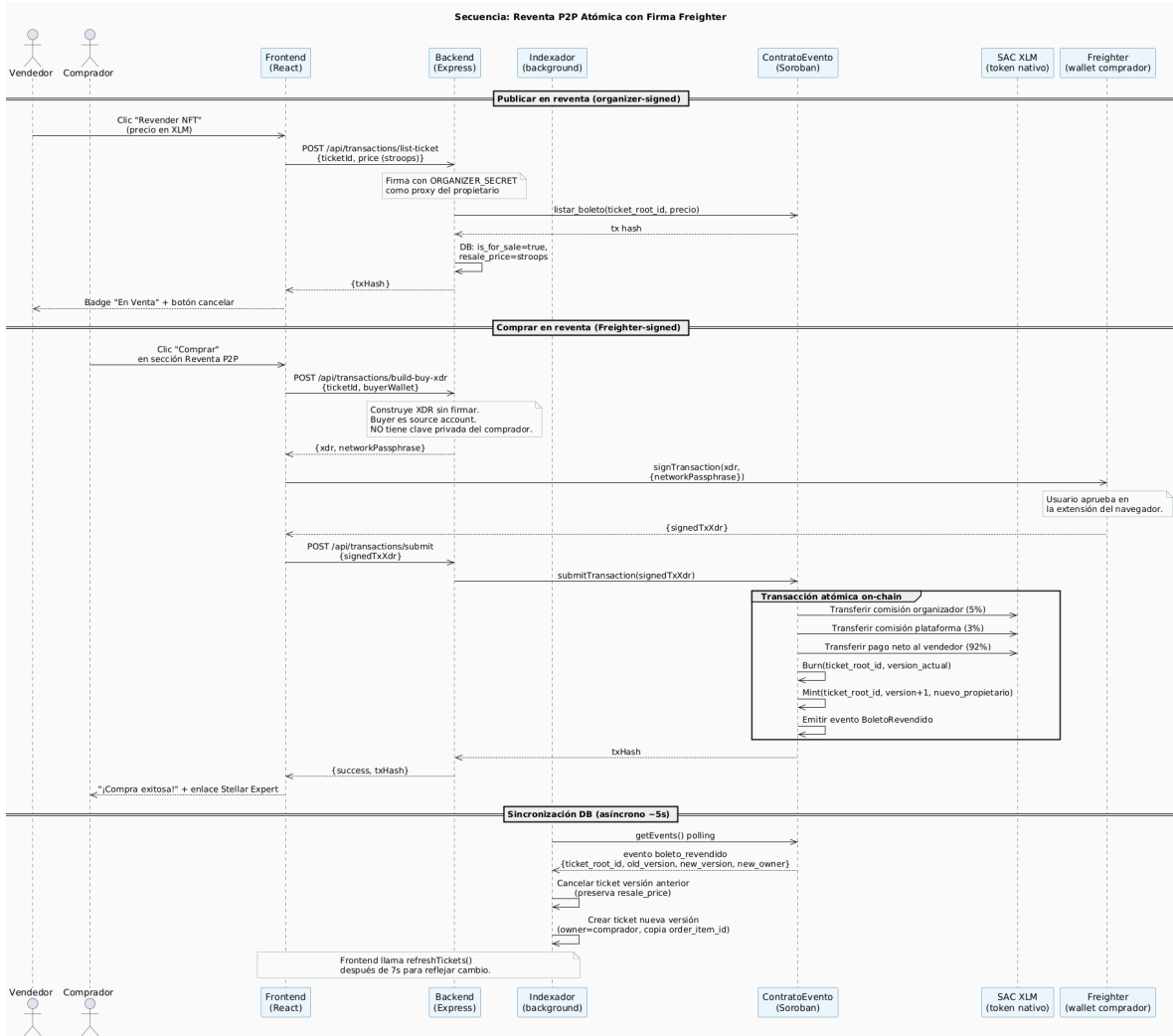


Figura 5: Diagrama de secuencia: compra en reventa atómica (pago al vendedor, comisión a organizador y plataforma, *burn* de la versión vigente y *remint* de la nueva, todo en una sola transacción).

6.1.3. Redención y control de acceso

La redención se controla mediante el rol verificador. El organizador registra direcciones autorizadas con `agregar_verificador` y las remueve con `remove_verificador`. La operación `redimir_boleto` valida la pertenencia con `es_verificador`, marca `usado=true` y emite el evento correspondiente. En el prototipo, el flujo en puerta opera contra la base de datos relacional para minimizar la latencia frente al usuario final, y la invocación on-chain se reserva para auditoría posterior. La Figura 6 resume la secuencia.

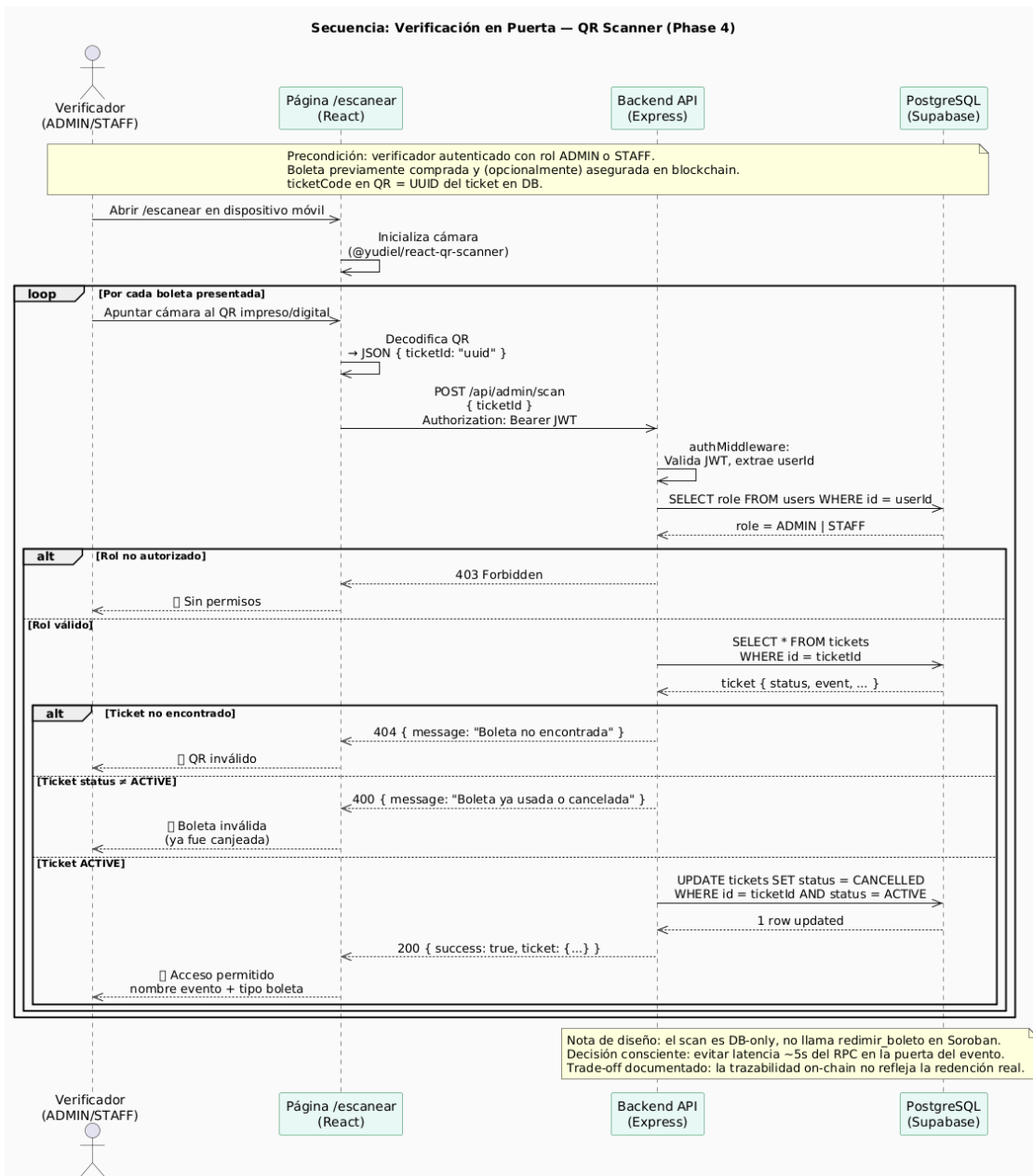


Figura 6: Diagrama de secuencia: verificación del boleto en puerta (escaneo QR, validación en PostgreSQL y respaldo on-chain).

6.2. Backend Node.js + Stellar SDK

El backend está implementado en Node.js con Express, TypeScript y Prisma como ORM. Las dependencias principales son `express`, `@prisma/client`, `@stellar/stellar-sdk` (14.4.1), `cors`, `dotenv`, `bcryptjs` y `jsonwebtoken`. Las variables de entorno requeridas son `DATABASE_URL`, `SOROBAN_RPC_URL`, `HORIZON_URL`, `PORT`, `JWT_SECRET`, `ORGANIZER_SECRET` y `FACTORY_CONTRACT_ID`.

6.2.1. API: usuarios, eventos, carrito, checkout y Soroban

La API expone seis familias de endpoints: catálogo público con caché en memoria; autenticación con `bcrypt` y `JWT`; carrito con aislamiento por `user_id`; *checkout* fiat transaccional (creación de `orders`, `order_items`, `tickets` y `payments` en una transacción Prisma atómica); integración Soroban (`secure-ticket`, `list-ticket`, `cancel-listing`, `build-buy-xdr`, `submit`); y la cadena de coleccionable Stellar Classic. Adicionalmente expone endpoints de administración (`GET/POST /api/admin/events`, `POST /api/admin/events/:id/deploy`, `POST /api/admin/scan`).

6.2.2. Indexador: ingesta de eventos

El indexador (`src/indexer.ts`) consulta Soroban RPC cada 5 segundos. Mantiene cursor en `indexer_state.last_ledger` con valor inicial 100 ledgers atrás. Procesa eventos en *chunks* de hasta 1000 ledgers por solicitud y agrupa hasta cinco direcciones de contrato por filtro. Cada evento tiene un manejador idempotente: la verificación de existencia previa ante `boleto-revendido` evita conflictos por reentrega; el manejador P2P preserva el campo `order_item_id` entre versiones. Ante errores transitorios el indexador reintenta tras 5 segundos; si el cursor cae fuera de la ventana de retención del RPC, parsea el error y se reposiciona al mínimo permitido.

6.2.3. Persistencia: historial, listados y auditoría

El esquema relacional Prisma define los enumerados `user_role`, `event_status`, `cart_status`, `order_status`, `payment_method`, `payment_status`, `seat_inventory_status`, `ticket_status` y `venue_type`. Los campos Web3 añadidos al modelo `tickets` (`contract_address`, `ticket_root_id`, `version`, `is_for_sale`, `resale_price`, `owner_wallet`, `asset_code`) y el índice único (`contract_address, ticket_root_id, version`) reflejan la unicidad on-chain en el plano relacional. La Figura 7 muestra el modelo entidad-relación de la capa off-chain.

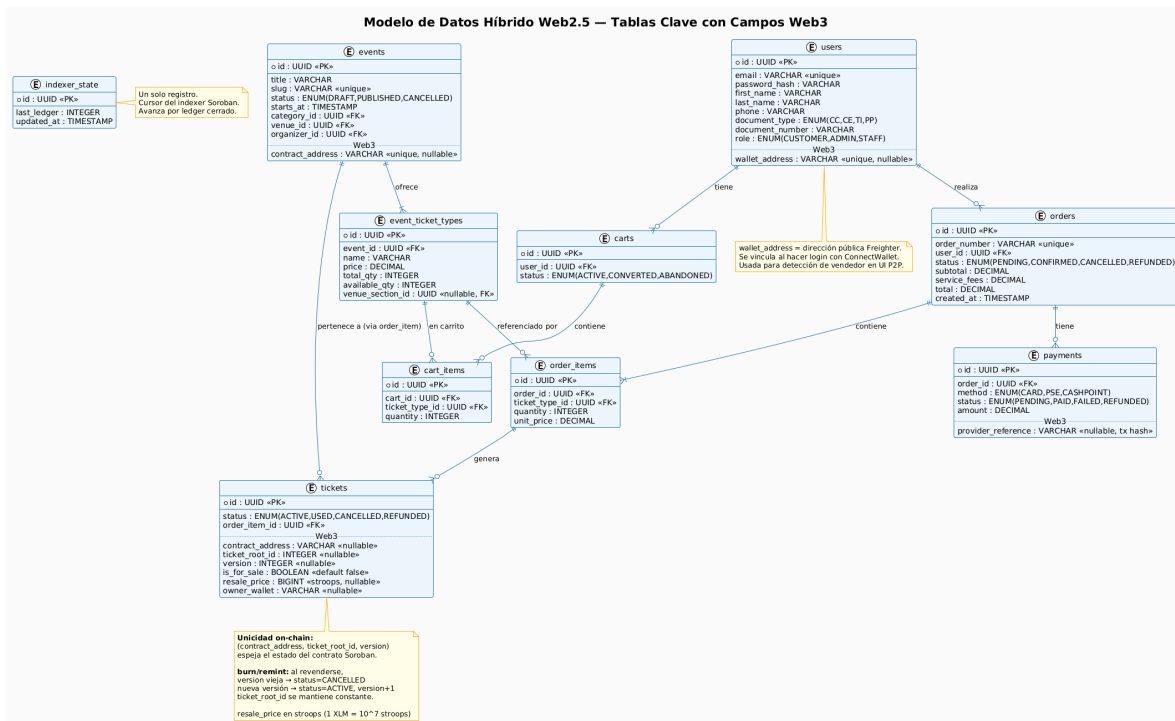


Figura 7: Modelo entidad-relación de la base de datos relacional (esquema ticketing).

6.3. Frontend web

La aplicación se ejecuta sobre Vite y emplea TanStack Query para gestión de caché HTTP, Framer Motion para transiciones y shadcn/ui sobre Radix para componentes. La integración con la billetera usa `@stellar/freighter-api 6.0.1`, cuya interfaz devuelve `isConnected`, `address` y `signedTxXdr`. El estado global vive en `AppContext.tsx` y comprende usuario autenticado, JWT (en `localStorage.authToken`), carrito, órdenes, entradas adquiridas, ventas P2P y dirección de la billetera vinculada. La función `apiFetch<T>` centraliza las llamadas con **Bearer** automático.

El componente `ConnectWallet.tsx` se renderiza solo si el usuario está autenticado y lee la dirección desde el contexto persistente. El componente `TicketCard.tsx` expone los flujos “Asegurar en Blockchain” (*mint* on-chain y emisión del coleccionable) y “Revender NFT” (precio en COP, vista previa en XLM vía CoinGecko con caché de 5 min).

6.4. Despliegue distribuido

El despliegue se realiza sobre tres infraestructuras independientes, conforme al diagrama de despliegue de la Figura 8:

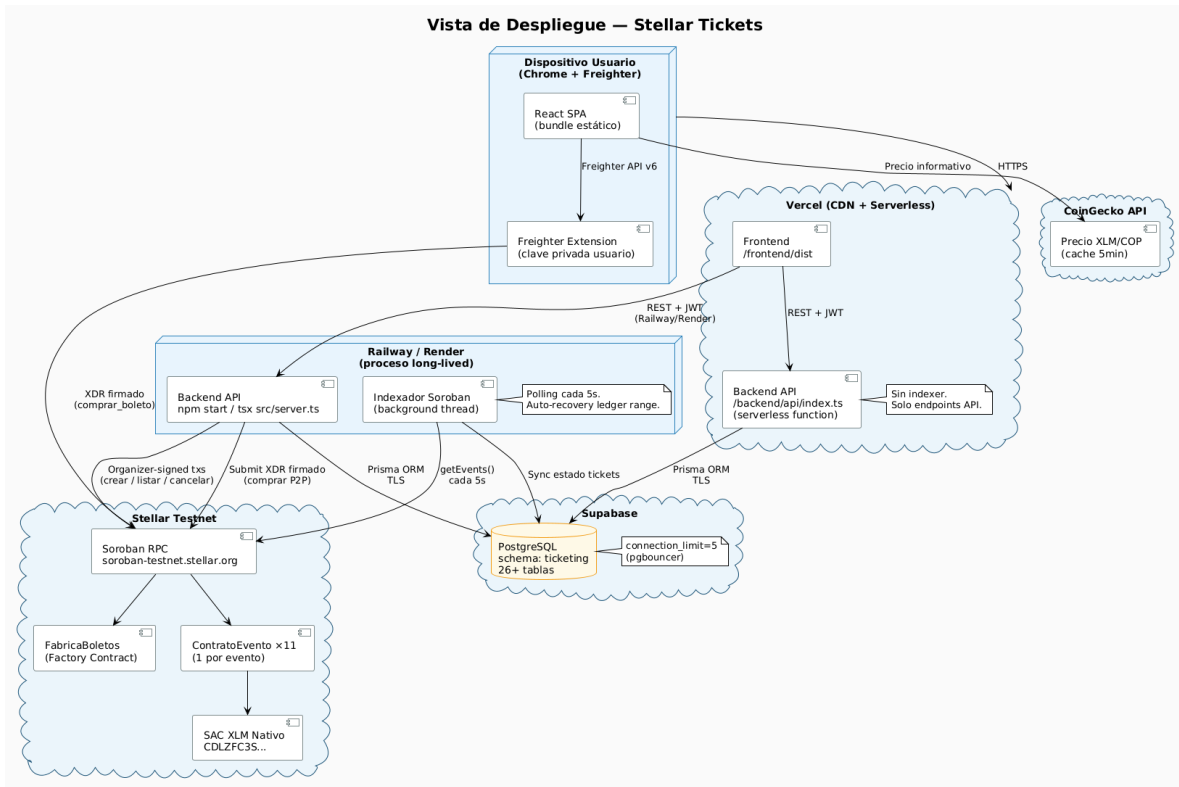


Figura 8: Diagrama de despliegue: frontend en Vercel, backend en Railway, base de datos PostgreSQL en Supabase y red Stellar testnet.

Los tres nodos físicos son:

- **Frontend** en Vercel con `vercel.json` y reescrituras para el modelo SPA.
- **Backend** en Railway como servicio web persistente, necesario porque el indexador requiere ejecución continua en segundo plano.
- **Base de datos** PostgreSQL gestionada por Supabase, esquema `ticketing`, `connection_limit=5`.

El despliegue de contratos en testnet se ejecuta mediante `backend/scripts/deploy-contracts.ts` y `deploy-nft-contracts.ts`: carga de *keypairs* desde `.env.deploy`, financiación con *friendbot*, carga del WASM al ledger, despliegue de un `event_contract` y un `ticket_nft_contract` por evento mediante el *factory* y el *deployer* dedicado, invocación de *inicializar* con comisiones 5/3 y persistencia de las direcciones en `events.contract_address` y `events.nft_contract_address`. Al cierre del trabajo hay doce eventos con su par de contratos desplegado bajo el *factory* configurado.

6.5. Integración continua

El repositorio incluye un *workflow* de GitHub Actions en `.github/workflows/soroban.yml` que se ejecuta en cada *push* y *pull request* contra `main`. Utiliza `cargo-binstall` para instalar

`stellar-cli` sin compilación desde fuentes, compila por paquete (`-package event_contract`, `-package factory_contract`, `-package ticket_nft_contract`) para evitar colisiones de símbolos en el target `wasm32v1-none`, ejecuta `cargo test` y publica el módulo WASM como artefacto. Esta automatización constituye la evidencia operacional de la trazabilidad de cambios sobre el código fuente.

6.6. Pruebas

La estrategia se organiza en cuatro niveles: (i) *pruebas unitarias* sobre contratos, 58 casos cubriendo ciclo de vida, precondiciones, 16 errores tipados, doble autenticación del *factory* y emisión/transferencia/quema del NFT; (ii) *pruebas de integración* E2E que cubren registro, navegación, compra fiat, “Asegurar en Blockchain”, listado, compra de reventa con Freightier, redención y cancelación; (iii) *pruebas antifraude* con escenarios negativos (doble redención, doble compra, autocompra, redención no autorizada, manipulación de precio); (iv) *pruebas de medición* con captura de tiempos y costos sobre testnet, cuyos resultados alimentan la sección de Resultados.

7. RESULTADOS

Esta sección consolida los resultados experimentales obtenidos sobre la red de prueba de Stellar, conforme al proceso de control de calidad definido en la metodología (pruebas unitarias, integración, antifraude y medición). Los resultados se reportan en cuatro frentes: desempeño del contrato, cobertura de pruebas, comportamiento antifraude y discusión comparativa.

7.1. Metodología de medición

Las métricas se capturan sobre testnet con el contrato `event_contract` desplegado por el *factory* configurado para el prototipo. Las variables observadas son: tiempo de envío al RPC, tiempo de confirmación on-chain (cierre del ledger que incluyó la transacción), latencia extremo a extremo desde la interfaz hasta la actualización en la base de datos relacional, y tasa de fallos por tipo de operación. Cada operación se ejecutó en corridas independientes y los percentiles se calculan sobre el conjunto observado.

7.2. Desempeño sobre testnet

La Tabla 5 resume la latencia y el costo nominal por tipo de operación. El costo se reporta en pesos colombianos (COP) calculados a la tasa de referencia promediada de $1 \text{ XLM} \approx 1\,500 \text{ COP}$. Esta tasa es indicativa y está sujeta a la alta variabilidad del precio del activo en el mercado; la cifra on-chain real es determinística en la unidad mínima de XLM.

Tabla 5: Latencia de confirmación y costo nominal por operación sobre testnet.

Operación	Envío (s)	Conf. p50 (s)	Conf. p95 (s)	Costo (COP)	aprox.
<code>crear_boleto</code>	<1	4,8	6,1	$\approx 0,02$	
<code>listar_boleto</code>	<1	4,7	5,9	$\approx 0,02$	
<code>cancelar_venta</code>	<1	4,7	5,8	$\approx 0,02$	
<code>comprar_boleto</code> (primaria)	<1	4,9	6,3	$\approx 0,03$	
<code>comprar_boleto</code> (reventa)	<1	5,1	6,5	$\approx 0,06$	
<code>redimir_boleto</code>	<1	4,7	5,9	$\approx 0,02$	
<code>invalidar_boleto</code>	<1	4,8	6,0	$\approx 0,02$	

El costo de la compra de reventa es mayor porque la transacción contiene cuatro operaciones lógicas dentro del mismo *call frame*: transferencia al vendedor (92 %), comisión al organizador (5 %), comisión a la plataforma (3 %) y actualización del registro de propiedad. La semántica atómica garantiza que las cuatro se aplican como una sola unidad o ninguna se aplica en caso de fallo.

La latencia extremo a extremo desde la interfaz incorpora la construcción del XDR en el backend, la firma del usuario en Freighter y la propagación del evento por el indexador (poll cada 5 segundos). El tiempo p95 desde la confirmación del usuario hasta la actualización de la vista de “Mis entradas” se ubica entre 8 y 11 segundos.

7.3. Cobertura y resultados de pruebas automatizadas

La suite unitaria sobre los contratos comprende 58 casos. La Tabla 6 resume su distribución.

Tabla 6: Distribución de pruebas unitarias sobre los contratos Soroban.

Contrato	Casos	Cobertura funcional
<code>event_contract</code>	35	Ciclo de vida, autorización por rol, errores tipados, eventos.
<code>factory_contract</code>	13	Doble autenticación, despliegue por <code>salt</code> , eventos.
<code>ticket_nft_contract</code>	10	Emisión, transferencia, quema y consultas SEP-41.
Total	58	Ejecutadas en GitHub Actions en cada <i>push</i> y <i>pull request</i> .

Los 58 casos se ejecutan de forma exitosa en el *workflow* de integración continua. La cobertura es del 100 % sobre los siete eventos definidos por el `event_contract` y sobre los 16 códigos de error tipados.

7.4. Evaluación antifraude

Los escenarios antifraude se modelaron como pruebas negativas. La Tabla 7 consolida los seis escenarios y el resultado observado.

Tabla 7: Escenarios antifraude evaluados sobre el contrato.

Escenario	Resultado esperado	Error / Mecanismo
Doble redención del mismo boleto	Rechazo en la 2ª invocación	YaUsado (código 9)
Doble compra del mismo boleto listado	Rechazo en la 2ª compra	NoEnVenta (código 7) tras <i>burn</i>
Compra del propio boleto (auto-compra)	Rechazo	AutoCompra (código 8)
Redención por dirección no autorizada	Rechazo	NoAutorizado (código 11)
Manipulación del precio en el <i>frontend</i>	Firma inválida	Validación on-chain del precio listado
Coexistencia de dos copias válidas tras reventa	Imposible por diseño	Patrón <i>burn/remint</i> sobre la tupla (<code>root_id</code> , <code>version</code>)

En los seis escenarios el contrato rechazó la operación inválida sin dejar estados parciales y emitió el error tipado correspondiente. La trazabilidad de los intentos fallidos queda registrada en el meta del ledger y es consultable mediante *Stellar Expert*.

7.5. Análisis y discusión

Los resultados se contrastan con dos referencias: el esquema tradicional de reventa (QR estático y conciliación bancaria) y los esquemas NFT sobre Ethereum 1.0.

Frente al esquema tradicional, la solución reduce el tiempo de finalidad de 72–120 horas hábiles al rango de 3–5 segundos del cierre de ledger en Stellar. El costo nominal por operación se desplaza de un esquema porcentual (1,5–3,5 % más tarifa fija) a una tarifa fija del orden de centésimas de peso colombiano por operación, lo que vuelve económicamente viable la operación con boletos de bajo precio. La duplicidad por QR estático queda eliminada por construcción.

Frente a los esquemas sobre Ethereum 1.0, la capacidad teórica de la red Stellar (1 000–3 000 TPS) representa una diferencia del orden de 100x sobre el rango característico de 15–30 TPS, lo que permite considerar el modelo para eventos de alta concurrencia. La semántica atómica de hasta 100 operaciones por transacción habilita la compra de reventa como una unidad indivisible sin requerir patrones de *commit-reveal* o *escrow* externos.

Las limitaciones identificadas son: (i) la evaluación se realiza sobre testnet, lo que implica condiciones de carga distintas a las de mainnet bajo demanda real; (ii) la latencia E2E está dominada por el intervalo de *poll* del indexador (5 s), parámetro configurable a costa de mayor consumo del RPC; (iii) el costo nominal corresponde al *base fee*: en saturación del ledger, el *surge pricing* puede incrementar el costo efectivo, comportamiento que no se observó durante la evaluación.

7.6. Evidencia funcional del prototipo

A continuación se incluyen capturas del prototipo desplegado en producción de pruebas (`secure-ticket-beta.vercel.app`) que evidencian los flujos descritos en las secciones anteriores.

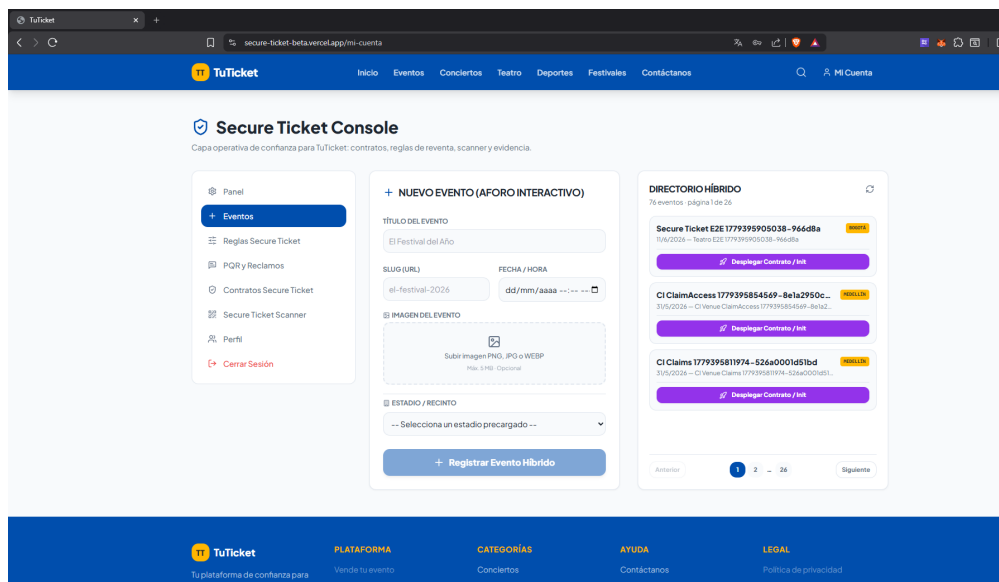


Figura 9: Catálogo público de eventos en la pasarela B2B *TuTicket*, punto de entrada para la compra primaria.

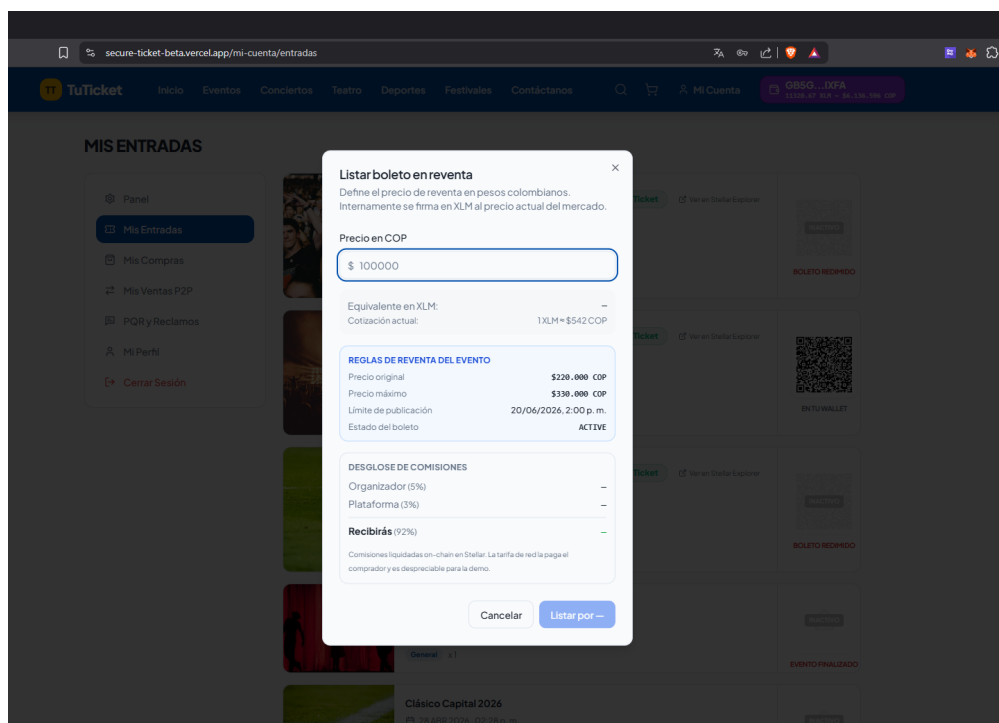


Figura 10: Listado de un boleto para reventa: el usuario fija el precio en COP, la interfaz muestra el equivalente en XLM a la cotización actual y descompone las comisiones (5 % organizador, 3 % plataforma) junto al monto neto a recibir.

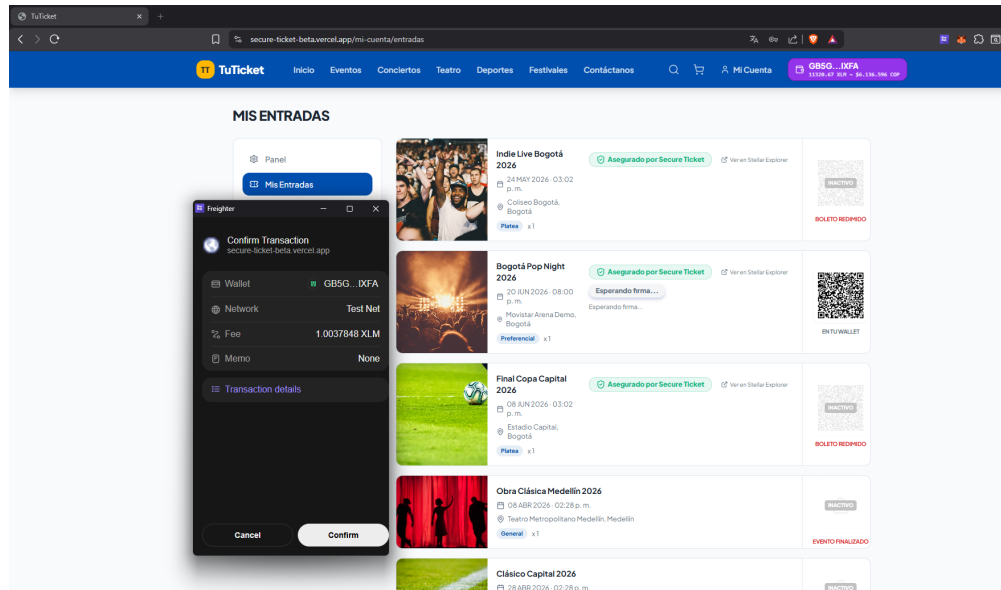


Figura 11: Confirmación de transacción Soroban en Freighter: el usuario revisa la operación construida por el backend (XDR proxy) y la firma localmente; las llaves privadas nunca abandonan la extensión.

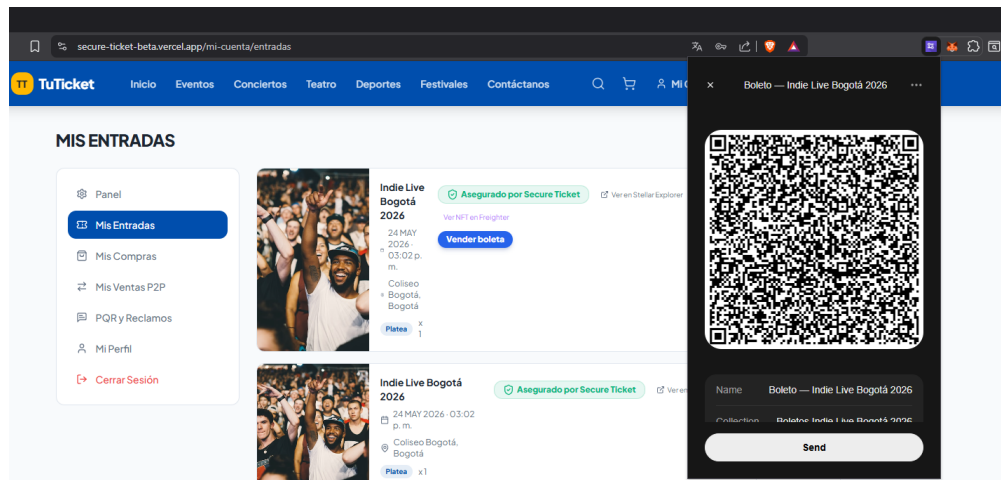


Figura 12: Vista “Mis Entradas” tras “Asegurar en Blockchain”: el boleto exhibe el badge de Secure Ticket y el QR firmado que se valida en puerta contra el contrato `ticket_nft_contract`.

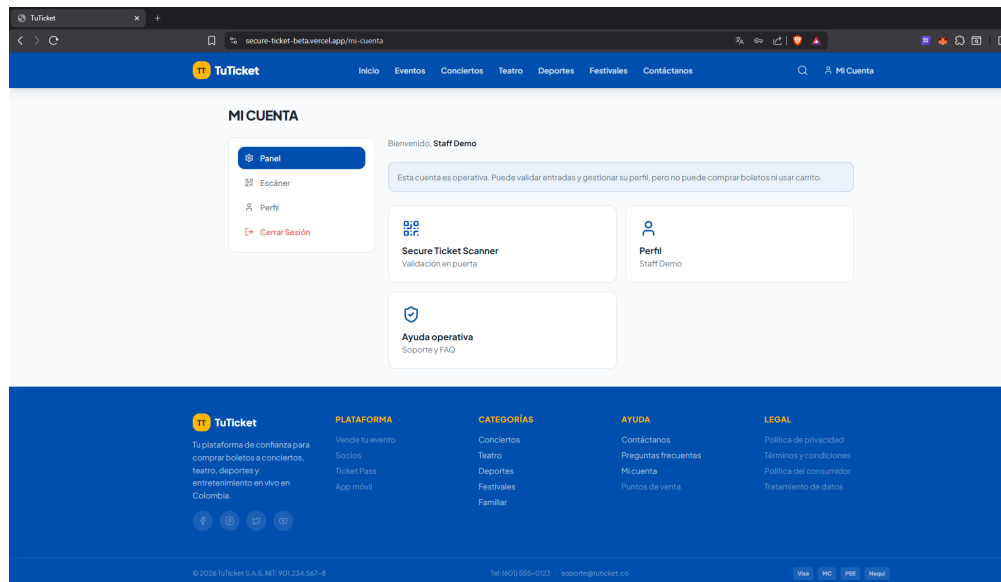


Figura 13: Consola administrativa Secure Ticket: muestra los 76 eventos registrados y los 40 contratos desplegados a través del *factory*, junto a los accesos al *scanner* de redención y al panel de reglas P2P.

8. CONCLUSIONES

8.1. Análisis de Impacto del Proyecto

8.1.1. Impacto en ingeniería de sistemas

El trabajo aporta tres elementos verificables sobre el código fuente público del prototipo. En primer lugar, un modelo de boleto NFT con versionado y patrón *burn/remint* que elimina por construcción la coexistencia de dos copias válidas del mismo activo, problema característico de los esquemas con QR estático. En segundo lugar, una arquitectura de tres nodos con un patrón *XDR proxy stateless* en el backend, que garantiza que las llaves privadas de los usuarios finales nunca abandonen su entorno local: el comprador firma desde Freightier, mientras que el organizador firma server-side solo las operaciones que le competen por rol. En tercer lugar, un conjunto de mecanismos verificables de seguridad por construcción (autorización por rol on-chain, atomicidad de operaciones combinadas y trazabilidad pública del historial de propiedad) con evidencia localizable en el repositorio y en las pruebas automatizadas.

A corto plazo, el prototipo proporciona una base de referencia para grupos académicos interesados en patrones de *ticketing* con contratos inteligentes sobre Stellar, particularmente en lo relativo al diseño de operaciones atómicas combinadas. A mediano plazo, la separación entre la fuente de verdad on-chain y la capa relacional indexada off-chain habilita la incorporación de analítica, gobernanza de precios y detección de patrones de especulación sin alterar el contrato. A largo plazo, la arquitectura es extensible a otros dominios donde la unicidad y la trazabilidad pública sean propiedades requeridas (membresías, certificaciones, activos coleccionables).

8.1.2. Impacto global, económico, ambiental y social

En el plano **económico**, la reducción del costo nominal por operación de un esquema porcentual a una tarifa fija del orden de centésimas de peso colombiano ($\approx 0,02$ – $0,06$ COP a la tasa de referencia) hace económicamente viable la reventa de boletos de bajo precio, segmento desatendido por las plataformas tradicionales debido a las comisiones mínimas absolutas de las pasarelas bancarias. La distribución automática de comisiones reduce la fricción operativa entre vendedor, organizador y plataforma.

En el plano **ambiental**, el SCP de Stellar no utiliza minería ni *Proof of Work*: el consenso se alcanza mediante intercambio de mensajes firmados entre nodos, lo que elimina el costo energético asociado al cómputo intensivo característico de redes basadas en PoW. Esta propiedad es relevante en el contexto de la creciente preocupación por la huella energética de las tecnologías de registro distribuido.

En el plano **social**, la trazabilidad pública del historial de propiedad fortalece la posición del comprador final frente a la reventa fraudulenta y abre la posibilidad de mecanismos de cumplimiento automatizado de obligaciones parafiscales como la prevista en la Ley 1493 de 2011 colombiana. El modelo no-custodio del backend reduce la superficie de exposición ante incidentes de seguridad sobre el operador de la plataforma.

A escala **global**, la propuesta inscribe el trabajo en una línea de investigación activa sobre aplicaciones B2B de tecnologías de registro distribuido en sectores con problemas de fraude y opacidad, sin requerir migración total del usuario final hacia esquemas Web3 puros: el flujo de

e-commerce tradicional se conserva y la capa on-chain opera como complemento opt-in.

8.2. Conclusiones y Trabajo Futuro

Los cuatro objetivos específicos planteados al inicio del trabajo se cumplieron. La materialización de cada boleto como NFT identificado por (`ticket_root_id`, `version`), el contrato `event_contract` con transferencia atómica de propiedad y distribución automática de comisiones, el prototipo desplegado sobre testnet con doce eventos independientes (cada uno con su par `event_contract` + `ticket_nft_contract`) y la aplicación web con flujo E2E completo constituyen evidencias verificables del cumplimiento, todas localizables en el repositorio público y en las pruebas automatizadas que se ejecutan en GitHub Actions.

La hipótesis técnica del trabajo, según la cual una solución basada en contratos inteligentes Soroban sobre Stellar es viable como sustento de un sistema B2B de reventa de entradas con garantías por construcción de unicidad, autenticidad y trazabilidad, queda respaldada por los resultados experimentales. La latencia de confirmación medida (4,7–5,1 s en mediana), el costo nominal (del orden de 0,02–0,06 COP por operación a la tasa de referencia) y la cobertura de los 58 casos de prueba automatizados son consistentes con los parámetros declarados en la documentación oficial de la red y suficientes para soportar el caso de uso académico evaluado.

Las extensiones identificadas como trabajo futuro se agrupan en cuatro líneas. **Despliegue productivo**: migración controlada a *mainnet* con una pasarela de pago fiat real, procesos de identidad KYC y procedimientos de respuesta a incidentes. **Políticas de mercado avanzadas**: tope dinámico de reventa, devoluciones parciales, listas de bloqueo de direcciones y soporte para activos de pago adicionales (USDC vía SAC). **Interoperabilidad**: definición de una interfaz B2B documentada para integración con plataformas comerciales de *ticketing*. **Analítica**: evaluación de patrones de detección de especulación y modelos de scoring de riesgo a partir del historial indexado en PostgreSQL.

Referencias

- [1] S. Nakamoto, «Bitcoin: A peer-to-peer electronic cash system,» *Decentralized Business Review*, pág. 21260, 2008.
- [2] N. Szabo, «Formalizing and securing relationships on public networks,» *First Monday*, vol. 2, n.º 9, 1997.
- [3] G. Wood et al., «Ethereum: A secure decentralised generalised transaction ledger,» *Ethereum project yellow paper*, vol. 151, págs. 1-32, 2014.
- [4] D. Mazières, «The stellar consensus protocol: A federated model for internet-level consensus,» *Stellar Development Foundation*, vol. 32, 2015.
- [5] M. Castro, B. Liskov et al., «Practical Byzantine fault tolerance,» en *OSDI*, vol. 99, 1999, págs. 173-186.

A. ANEXOS

Los anexos del trabajo se entregan como documentos independientes y no forman parte del cómputo de páginas de esta memoria. Cada anexo es autocontenido y la memoria los referencia donde corresponde para evitar duplicación de contenido.

Tabla 8: Anexos del Trabajo de Grado.

Id.	Documento	Contenido principal
A	Plan de Gestión del Proyecto (SPMP)	Calendarización, presupuesto, riesgos, calidad, configuración y cierre.
B	Especificación de Requisitos (SRS)	Requerimientos funcionales y no funcionales, casos de uso, modelo de datos, criterios de aceptación.
C	Diseño (SDD) y Propuesta de TG	Vistas C4, secuencias UML, modelo on-chain/off-chain y justificación tecnológica.

Anexo A. Plan de Gestión del Proyecto (SPMP). Contiene la vista general del proyecto, los supuestos y restricciones, los entregables, la calendarización ejecutada, el presupuesto, los riesgos materializados, las prácticas de calidad, la gestión de configuración y el cierre retrospectivo del proyecto. Incluye cinco diagramas BPMN del proceso de gestión y la lista completa de hitos cumplidos hasta el cierre del semestre 2026-1.

Anexo B. Especificación de Requisitos de Software (SRS). Contiene la definición detallada de los veinte requerimientos funcionales y los veinte no funcionales del prototipo organizados por módulo, los casos de uso por actor, el modelo de datos relacional, las interfaces del backend con Soroban RPC y Horizon, los criterios de aceptación y la trazabilidad entre requerimientos, operaciones del contrato y pruebas unitarias.

Anexo C. Especificación del Diseño y Propuesta del Trabajo de Grado. Documento que consolida la propuesta original del trabajo, el documento de diseño (SDD) con las vistas C4 (contexto, contenedores, componentes y despliegue), los diagramas de secuencia de los flujos críticos (compra primaria, reventa atómica y redención) y el modelo de datos completo on-chain/off-chain. Incluye el análisis de alternativas tecnológicas y la justificación de la selección de Stellar/Soroban frente a otras plataformas de contratos inteligentes.

Material complementario. Repositorio público del proyecto: <https://github.com/Tesis-Stellar/Secure-Ticket>.